

Datacenter Design and Operation

Appunti Sintetici e Rielaborati

A cura di Fabio Piscitelli M.715339

Anno Accademico 2025/2026

Indice

1	Introduzione, Architettura e Infrastruttura	1
1.1	Evoluzione: Dalle Sale Server alle AI Factories	1
1.1.1	Dai Mainframe alla Bolla delle Dot-Com	2
1.1.2	L'Era dell'Intelligenza Artificiale e la Top500	2
1.2	Fondamenti e Vincoli Fisici	2
1.2.1	Pavimento Flottante	3
1.3	Raffreddamento e Gestione Termica	3
1.3.1	L'Architettura ad Aria: Sistemi CRAC e Flussi	4
1.3.2	Liquid Cooling	5
1.4	L'Efficienza Energetica (PUE)	5
1.5	Sicurezza e Continuità	6
2	Cablaggio, Switching e Fabric di Rete	7
2.1	Il Mezzo Fisico: Rame e Fibra Ottica	7
2.1.1	Cavi in Rame ed Ethernet	7
2.1.2	Fibra Ottica e Transceiver	8
2.2	Architettura interna degli Switch	9
2.2.1	Software-Defined Networking (SDN) e SONiC	10
2.3	Topologie di Rete: Dal 3-Tier allo Spine-and-Leaf	13
2.4	Segmentazione e Virtualizzazione della Rete	15
2.4.1	Jumbo Frames e MTU	15
2.4.2	VLAN (Virtual LAN)	15
2.4.3	VXLAN e Overlay	15
2.5	Fabric ad Alte Prestazioni (HPC e AI)	16
3	Storage: Architetture e Servizi	17
3.1	Evoluzione del Mezzo di Memorizzazione	17
3.1.1	Dal Nastro Magnetico all'HDD e all'SSD NVMe	17
3.1.2	Celle NAND e Asimmetria	18
3.1.3	Misure di Performance: IOPS, Latenza e Bandwidth	18
3.2	Aggregazione e Protezione: RAID e Resilienza	19
3.3	Architetture di Storage nel Datacenter	19
3.3.1	SAN (Storage Area Network)	19
3.3.2	NAS (Network Attached Storage)	20
3.3.3	Object Storage	20
3.3.4	HCI (Hyper-Converged Infrastructure)	21
3.3.5	Storage per l'AI: Vector Database	21
3.4	Servizi Avanzati e Resilienza	21

3.4.1	Efficienza: Deduplicazione, Compressione e Tiering	21
3.4.2	Snapshot, Thin Provisioning e Replica	22
3.4.3	Backup vs Replica	22
4	Compute e Virtualizzazione	24
4.1	Architettura Compute: CPU, GPU e NPU	24
4.1.1	Sostenibilità Energetica	24
4.1.2	Architettura CPU: Pipeline e Multi-Core	24
4.1.3	GPU e NPU: Il cambio di paradigma	26
4.2	Architetture Server Fisici e BMC	26
4.2.1	Server Rack (1U / 2U)	27
4.2.2	Blade Server	28
4.2.3	Server HGX (AI GPU Server)	29
4.2.4	Standardizzazione Hyperscaler: Open Compute Project (OCP)	29
4.3	Virtualizzazione e Live Migration	30
4.3.1	L'Evoluzione dell'Hypervisor: Dai Ring CPU alla Virtualizzazione Hardware	30
4.3.2	Astrazione dell'Hardware: vCPU, Pause e Resume	31
4.3.3	Il Virtual Switch (vSwitch)	31
4.3.4	Live Migration	31
4.4	Containerizzazione	32
4.4.1	Meccanismi di Isolamento e Vulnerabilità	32
4.4.2	Dettagli Architetture Docker	32
4.4.3	Network: VM vs Container	33
5	Cloud Computing, Orchestrazione e Sicurezza	34
5.1	Il Modello Cloud Computing	34
5.1.1	Definizione e Caratteristiche (NIST)	35
5.1.2	Modelli di Servizio	35
5.2	Architettura di Riferimento (Reference Model)	36
5.2.1	Cloud Service Lifecycle	36
5.3	Business Continuity e Data Protection	37
5.3.1	Tecniche di Resilienza e Limiti Geografici	37
5.3.2	Application Resiliency: Progettare per il Fallimento	38
5.4	Sicurezza (Security)	38
5.4.1	Minacce Classiche e Attacchi	39
5.4.2	Gestione delle Identità e Policy (AAA e RBAC)	39
5.4.3	Approccio Defense-in-Depth e Zero Trust	39
5.4.4	Gestione dei Rischi (GRC)	40
6	Operations e Considerazioni Finali	41
6.1	Service Operations Management	41
6.1.1	Monitoring e Predictive Failure	41
6.1.2	Capacity Monitoring e Management	42
6.2	Gestione dei Cambiamenti e dei Guasti	42
6.2.1	Change Management	42
6.2.2	Incident vs. Problem Management	42
6.3	Considerazioni Finali sul Datacenter	43

Capitolo 1

Introduzione, Architettura e Infrastruttura

1.1 Evoluzione: Dalle Sale Server alle AI Factories

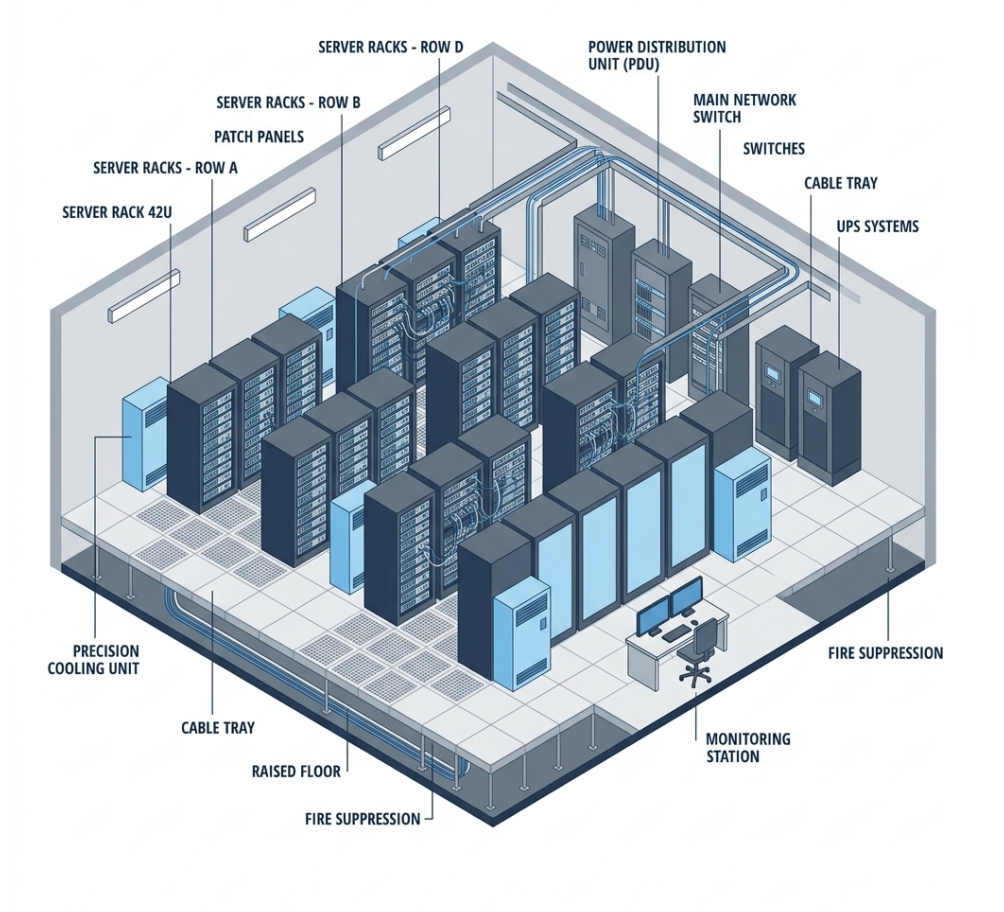


Figura 1.1: Illustrazione isometrica di un moderno datacenter ad alta densità.

1.1.1 Dai Mainframe alla Bolla delle Dot-Com

Storicamente, il calcolo centralizzato nasce negli anni '60 e '70 con i **Mainframe** (es. sistemi IBM): giganteschi armadi chiusi in stanze asettiche ("glass house") a cui gli utenti accedevano tramite terminali stupidi (green screen). In questa fase, il sistema era un blocco monolitico proprietario e altamente centralizzato. Negli anni '80 e '90 si passa al modello **Client-Server**, con i server x86 (architettura aperta) distribuiti in semplici "sale server" aziendali, spesso ex ripostigli riadattati con un condizionatore commerciale e nessuna vera progettazione ingegneristica. Alla fine degli anni '90, l'esplosione di Internet porta alla **bolla delle Dot-Com**: la necessità di scalare globalmente spinge le aziende ad affittare spazio (Colocation) in giganteschi edifici specializzati gestiti da telco e provider, per garantire l'accessibilità H24. Da qui nasce il vero e proprio concetto moderno di Datacenter.

1.1.2 L'Era dell'Intelligenza Artificiale e la Top500

Oggi si tratta di un'infrastruttura critica a livello nazionale, che richiede investimenti dell'ordine di centinaia di milioni, se non miliardi, di euro. L'esplosione dell'intelligenza artificiale (IA) ha portato all'era delle **AI Factories**: complessi industriali in grado di assorbire fino a 1-2 Gigawatt, l'equivalente di un'intera città. La potenza di questi cluster è costantemente tracciata da due classifiche mondiali semestrali (nate nel 1993):

- **Top500**: Misura le prestazioni assolute (benchmark di algebra lineare in PetaFLOPS).
- **Green500**: Misura l'efficienza energetica (GigaFLOPS per Watt), evidenziando il profondo **impatto ambientale** di queste strutture.

L'IA ha radicalmente alterato i paradigmi del passato: mentre nel Cloud computing tradizionale si sfrutta la virtualizzazione per massimizzare l'uso dell'hardware (normalmente inattivo per il 90% del tempo), l'IA è il primo *workload* che **satura integralmente l'hardware fisico**. Questo rende la virtualizzazione inutile nello stack AI e impone l'utilizzo di architetture hardware distribuite su decine di migliaia di GPU interconnesse.

1.2 Fondamenti e Vincoli Fisici

La costruzione di un datacenter moderno si scontra con limiti ingegneristici definiti **Hard Thresholds**: soglie fisiche oltre le quali non è possibile semplicemente "scalare" (ingrandire) una tecnologia, ma diventa necessario un ripensamento radicale.

Attenzione: Scale Up vs Scale Out

- **Scale Out**: aggiungere nuovi nodi o server per distribuire il carico.
- **Scale Up**: aumentare la capacità del singolo nodo (o, nel caso elettrico, del singolo cavo o edificio).

L'infrastruttura elettrica di un intero datacenter è costretta a operare in logica **Scale Up**: all'aumentare del consumo occorrono cavi in rame via via più spessi e costosi.

1.2.1 Pavimento Flottante



Figura 1.2: Sollevamento di una piastrella dal pavimento flottante in un data center (Wikimedia Commons).

Una delle eredità ingegneristiche più diffuse è il **Floating Floor** (pavimento flottante). Costituito da pannelli sollevati su piedistalli metallici, nei moderni DC può reggere fino a 1.5 tonnellate per metro quadrato. I due grandi vantaggi sono:

1. **Gestione cavi:** passaggio ordinato dell'infrastruttura di cablaggio e tubazioni idrauliche.
2. **Plenum per l'aria:** creazione di un condotto pressurizzato sotto il pavimento per distribuire l'aria fredda dal sistema di raffreddamento verso le griglie posizionate di fronte ai rack.

1.3 Raffreddamento e Gestione Termica

A causa dell'estrema densità di calore, dovuta all'elevato **TDP (Thermal Design Power)** dei nuovi processori (che raggiungono il Kilowatt per singola GPU), il controllo termico è diventato l'aspetto più critico del design. La temperatura estrema può fondere i componenti in silicio; l'interruzione del sistema di raffreddamento, unita al tentativo delle ventole dei server di spingere aria, causa aumenti termici esponenziali che possono danneggiare l'intera stanza in pochissimi minuti.

1.3.1 L'Architettura ad Aria: Sistemi CRAC e Flussi

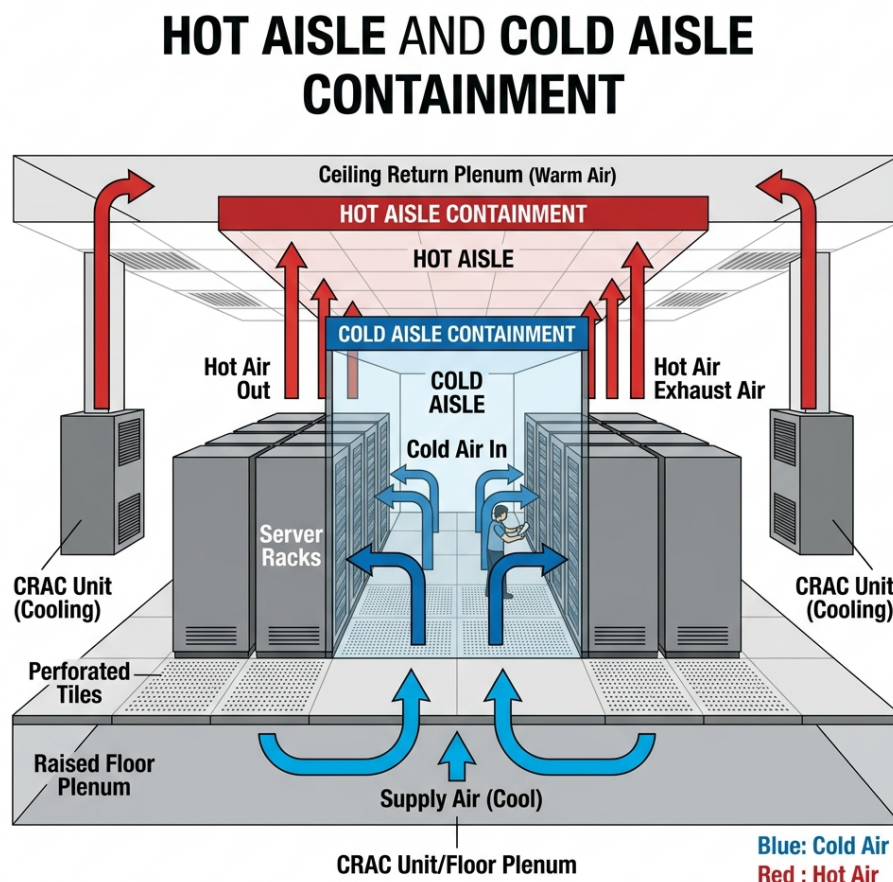


Figura 1.3: Schema del flusso d'aria: contenimento del corridoio freddo (Cold Aisle Containment) e corridoio caldo.

I sistemi **CRAC (Computer Room Air Conditioning)** si basano sul pavimento flottante e sul principio della rigorosa separazione tra *Cold Aisle* (corridoio freddo) e *Hot Aisle* (corridoio caldo). Il flusso dell'aria segue un percorso preciso: le unità CRAC immettono aria ghiacciata pressurizzata nel *plenum* (lo spazio sotto il pavimento flottante). L'aria fuoriesce dalle grate forate esattamente di fronte ai rack (Cold Aisle). Le ventole dei server risucchiano l'aria frontalmente, raffreddano i dissipatori interni, ed espellono l'aria rovente dal retro nel corridoio caldo (Hot Aisle), dove sale verso il soffitto per tornare ai CRAC ed essere raffreddata nuovamente tramite scambiatori di calore ad acqua (Chiller).

Regola Aurea

Mai mescolare l'aria calda e l'aria fredda. Se i flussi si mescolano, l'efficienza crolla e le CPU rischiano il *thermal throttling*. Per questo si usano porte a vetri e tettucci chiusi per creare sistemi di **contenimento fisico** dei corridoi.

Tuttavia, l'aria è un pessimo conduttore termico. Per carichi elevati (Rack oltre i 15-20 kW, per non parlare dei moderni rack AI da 200 kW), i sistemi CRAC ad aria sono insufficienti.

1.3.2 Liquid Cooling

Il raffreddamento a liquido è oggi uno standard di fatto per l'high performance computing (HPC):

- **Direct-to-Chip:** placche di rame raffreddate ad acqua poste direttamente a contatto con CPU, GPU e RAM. Rimuove la necessità di ventole ad alta velocità.
- **Waterless Cooling:** utilizzo di fluidi dielettrici (es. glicole) in tubazioni. In caso di sversamento, non causano cortocircuiti.
- **Immersion Cooling:** immersione completa della scheda madre in olio minerale. Elimina ogni punto di calore localizzato (hot-spot), ma rende la manutenzione complessa e costosa.

1.4 L'Efficienza Energetica (PUE)

L'efficienza di un datacenter si misura con un valore adimensionale standard, il **PUE** (**Power Usage Effectiveness**):

$$PUE = \frac{\text{Total Facility Power}}{\text{IT Equipment Power}}$$

Più il PUE si avvicina a 1, maggiore è l'efficienza (significa che non si sta sprecando energia per raffreddare). I migliori data center odierni hanno un PUE annuo attorno a 1.15 - 1.20.

Le Trappole del PUE

Sebbene il PUE sia lo standard de facto, presenta diverse ambiguità critiche che possono trarre in inganno:

- **Impatto della Stagionalità:** Il PUE non è un valore fisso. Un datacenter che sfrutta il raffreddamento ad aria esterna (Free Cooling) misurerà un PUE eccellente (es. 1.05) in inverno, ma vedrà crollare l'efficienza (es. 1.50) in estate a causa dell'attivazione forzata dei compressori meccanici.
- **Il Paradosso del Datacenter Vuoto:** Un data center appena inaugurato con pochissimi server accesi (IT Power molto basso) avrà paradossalmente un PUE pessimo. Questo perché i sistemi di raffreddamento industriali (chiller, pompe) hanno un consumo di base ineliminabile anche girando al minimo, sbilanciando la frazione matematica.
- **Valutazione Economica (Split Domestici):** Per raffreddare un piccolo locale CED, installare un impianto industriale (PUE 1.3) costerebbe decine di migliaia di euro e richiederebbe contratti di manutenzione costosi. Spesso si scelgono semplici condizionatori domestici "split": portano il PUE a un disastroso 2.0, ma risultano economicamente la scelta più sensata in termini di investimento iniziale e affidabilità pratica.

1.5 Sicurezza e Continuità

La progettazione logica segue la cosiddetta **CIA Triad**:

- **Confidentiality** (Riservatezza)
- **Integrity** (Integrità)
- **Availability** (Disponibilità)

L'approvvigionamento elettrico è vitale. La distribuzione avviene seguendo una rigorosa catena: Dalla rete esterna in alternata (AC), si passa agli **UPS (Uninterruptible Power Supply)** industriali (basati su enormi batterie e sistemi a molla precompressa per scambi istantanei) che stabilizzano la corrente e prevengono micro-interruzioni. Dagli UPS si arriva alle **PDU (Power Distribution Unit)** gerarchiche, fino ai server che internamente convertono l'AC in corrente continua (DC).

La necessità di resilienza estrema si scontra con limiti pratici e geografici. La collocazione dei data center deve infatti tener conto di variabili critiche quali rischio sismico, escursioni termiche, accesso immediato alla rete di trasmissione in altissima tensione e obblighi normativi imposti dal GDPR.

Capitolo 2

Cablaggio, Switching e Fabric di Rete

2.1 Il Mezzo Fisico: Rame e Fibra Ottica

La rete è il sistema vascolare del datacenter. L'evoluzione della densità di traffico ha segnato il progressivo abbandono del rame a favore della fibra ottica.

2.1.1 Cavi in Rame ed Ethernet

Storicamente, le reti informatiche affrontavano il problema delle collisioni sui cavi condivisi. Una topologia diffusa era il **Token Ring (IEEE 802.5)**, una rete ad anello fisico in cui un "gettone" elettronico veniva passato tra i computer: solo chi possedeva il gettone poteva trasmettere. Sebbene garantisse una latenza prevedibile, la rottura di un singolo cavo o nodo paralizzava l'intero anello. Ethernet ha vinto la battaglia commerciale grazie alla topologia a stella e al protocollo CSMA/CD, molto più robusto e flessibile ma pronò a inefficienze e collisioni ad alto carico prima dell'introduzione dello switching.

Sui cablaggi, i data center si basavano pesantemente sul rame (es. i vecchi cavi **Cat 5** o Cat 5e che limitavano le velocità a 100 Mbps o 1 Gbps). I limiti fisici del rame (diafonia e attenuazione del segnale) e la necessità di gestire un preciso *pin-out* per l'allineamento dei contatti hanno reso sempre più complessa l'evoluzione. Oggi, il tradizionale cavo in rame (standard Ethernet con connettore RJ45) è confinato al limite pratico dei **10 Gbps (Cat 6A)**. Andare oltre con standard come Cat 7 o Cat 8 risulta impraticabile: i cavi diventano spessi, rigidi e difficili da gestire (il cosiddetto *cable management* è vitale, poiché un cablaggio disordinato blocca il flusso d'aria e distrugge l'efficienza termica del rack).

2.1.2 Fibra Ottica e Transceiver



Figura 2.1: Connettori ottici in fibra (LC e SC a confronto).

La trasmissione ottica azzerla la dissipazione termica e aumenta vertiginosamente la banda, ma richiede l'uso di **transceiver** (moduli hot-swappable) per convertire il segnale elettrico in ottico. L'evoluzione di questi moduli ha definito le velocità del datacenter:

- **SFP28:** Standard per porte server a 25 Gbps (singola lane).
- **Famiglia QSFP (Quad SFP):** Usa 4 lane interne. Include QSFP+ (40G), **QSFP28** (100G, standard per uplink), fino ai moderni **QSFP56** (200G) e **QSFP-DD** (fino a 800G).

Il salto prestazionale oltre i 100G non è avvenuto aggiungendo cavi, ma cambiando modulazione del segnale da **NRZ** (1 bit per simbolo) a **PAM4** (2 bit per simbolo, raddoppiando la banda alla stessa frequenza). I moduli utilizzano codici colore standardizzati incisi sul corpo (es. blu per monomodale, nero/grigio per multimodale).

Le fibre si dividono in due grandi famiglie:

- **Multimodale (OM):** core più largo ($50\ \mu\text{m}$), permette più rimbalzi dei fotoni. Ha distanze operative brevi (fino a $\sim 100\text{-}300\ \text{m}$) ma costi inferiori. È lo standard dominante *all'interno* del datacenter.
- **Monomodale (OS):** core sottilissimo ($9\ \mu\text{m}$), permette un unico percorso rettilineo. Ideale per collegamenti metropolitani a lungo raggio (fino a 40 km).

2.2 Architettura interna degli Switch

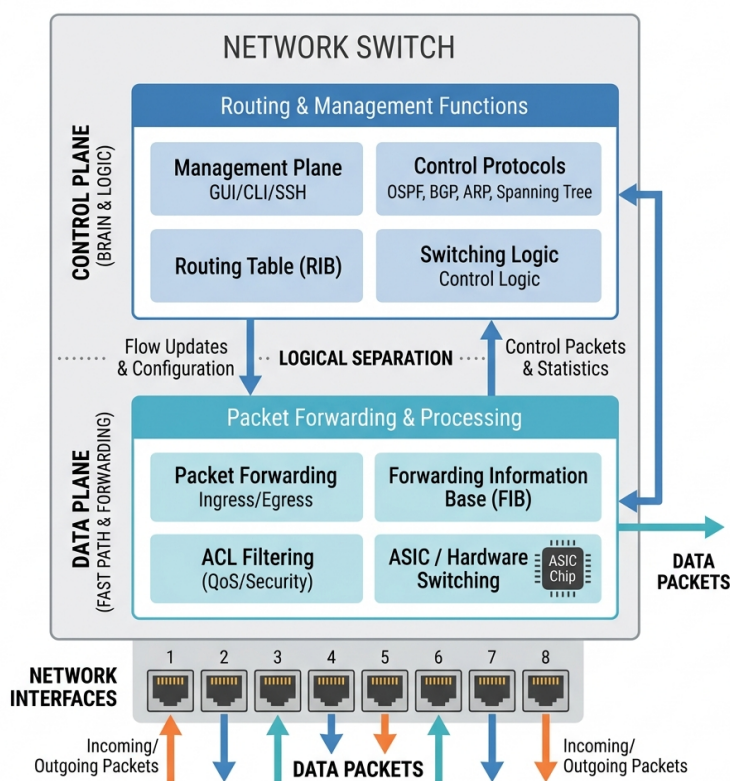


Figura 2.2: Architettura logica: separazione tra Control Plane e Data Plane in un dispositivo di rete.

In passato, l'architettura di rete dominante si basava su enormi *chassis modulari* proprietari (es. i grandi switch core di Cisco), costosi e difficili da scalare gradualmente. Lo switch moderno, spinto dalle esigenze iperscalabili del cloud, ha abbandonato gli chassis a favore del **Fixed Form Factor** (1U o 2U), spesso definito *White-box switch*.

2.2.1 Software-Defined Networking (SDN) e SONiC

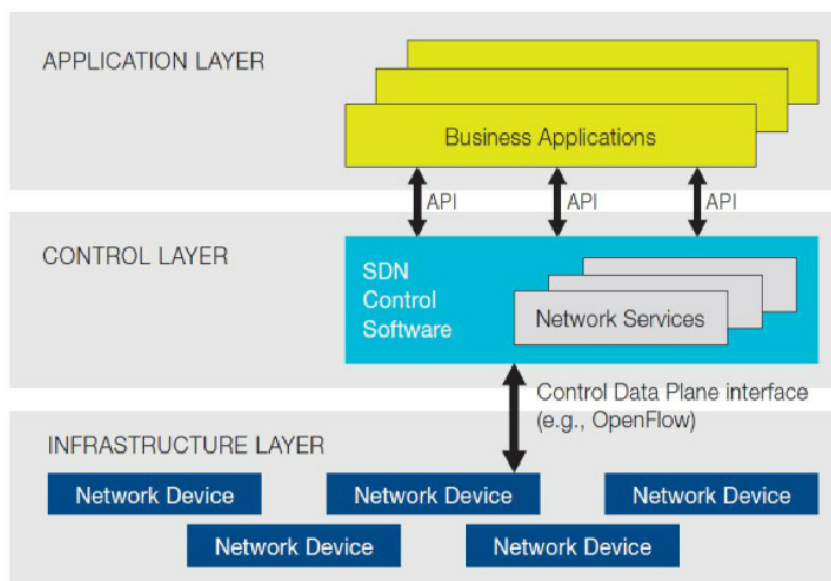


Figura 2.3: Modello SDN: l'intelligenza di rete viene astratta dall'infrastruttura fisica sottostante.

La grande rivoluzione è stata la separazione concettuale in due piani:

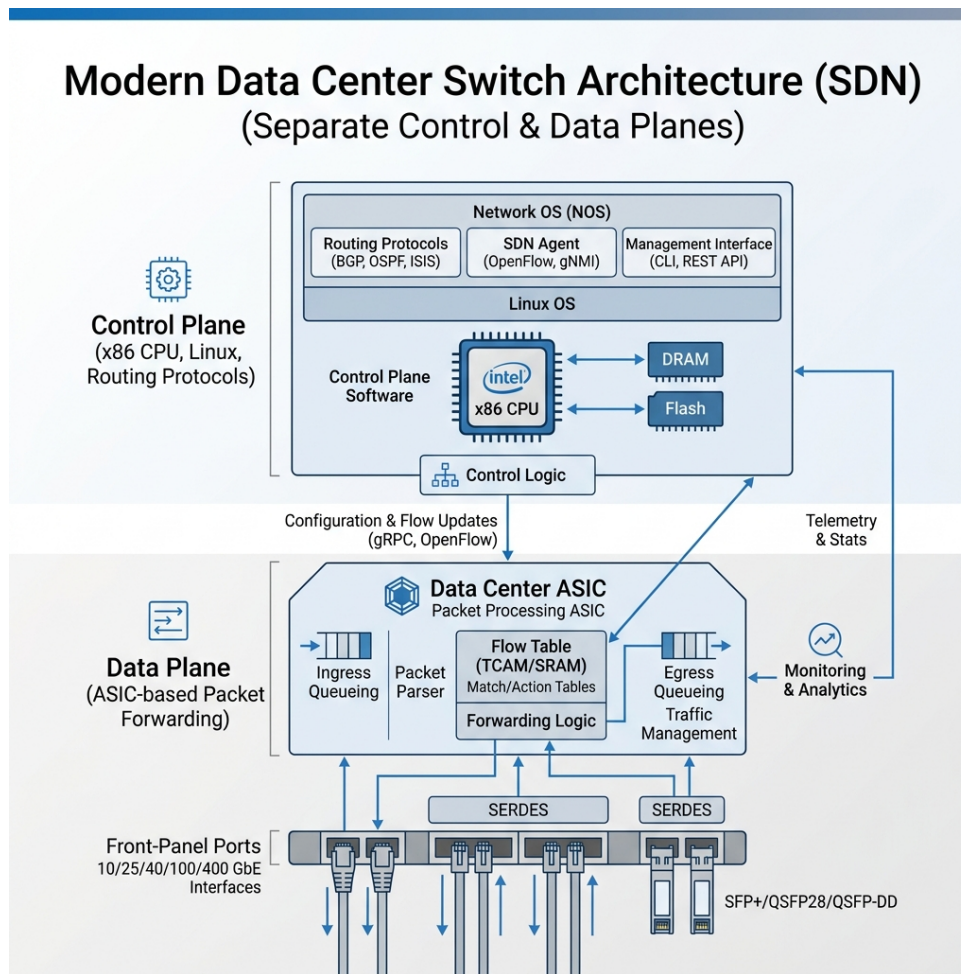


Figura 2.4: Dettaglio architetturale della separazione tra ASIC (Data Plane) e CPU x86 (Control Plane).

1. **Data Plane (Piano Dati):** hardware dedicato (spesso ASIC commodity, es. chip Broadcom) che inoltra i pacchetti (*forwarding*) in modo non-bloccante a velocità *line-rate*. L'introduzione di un frame in questo ASIC hardware aggiunge una latenza irrisoria di circa **600 ns**.
2. **Control Plane (Piano di Controllo):** un vero e proprio PC (spesso basato su processori **Intel Celeron** o simili) integrato nello chassis, che gestisce la logica di rete (BGP, OSPF). Essendo un PC Linux completo, è possibile compilarci software sopra (es. eseguire l'editor VSCode direttamente dentro lo switch per editare script).

Running vs Startup Config

Nei sistemi operativi di rete è vitale la gestione della memoria del Control Plane. Le modifiche ai parametri (es. via CLI) avvengono nella **Running Configuration** (sita nella RAM volatile); hanno effetto immediato ma spariscono in caso di spegnimento o blackout. Per renderle definitive devono essere salvate nella **Startup Configuration** (memoria flash/NVRAM) tramite il comando `write` (o `copy running-config startup-config`). Molti disservizi catastrofici (e impossibili da debuggare a posteriori) nascono da riavvii improvvisi di apparati le cui configurazioni non erano mai state persistite su disco.

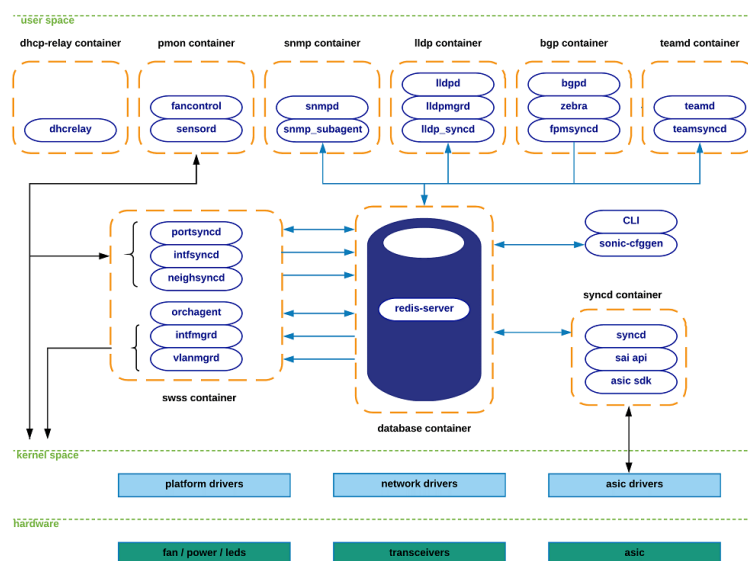


Figura 2.5: Architettura SONiC: ogni funzione di rete gira nel proprio container Docker comunicando tramite Redis.

La disaggregazione hardware-software ha portato alla nascita di Sistemi Operativi di Rete (NOS) open source come **SONiC** (Software for Open Networking in the Cloud, creato da Microsoft). Avviato tramite il bootloader ONIE, SONiC tratta lo switch come un server standard, eseguendo ogni demone di routing all'interno di *container Docker* isolati.

L'API OpenFlow e il Firewall Sandwich

Per istruire il Data Plane su come inoltrare i pacchetti, il Control Plane (o un controller centralizzato) utilizza protocolli di comunicazione, il più famoso dei quali è l'**API OpenFlow**. Questo permette di manipolare programmaticamente le *Flow Tables* all'interno dell'ASIC. Grazie a questa astrazione, si realizzano architetture come il **Firewall Sandwich**: il controller SDN (OpenFlow) istruisce lo switch per deviare tutto il traffico in ingresso verso una serie di firewall hardware posti in parallelo (il "ripieno" del sandwich). Una volta ispezionato, il traffico rientra nello switch che lo instrada alla destinazione finale, bilanciando il carico e garantendo scalabilità lineare della sicurezza, in modo trasparente.

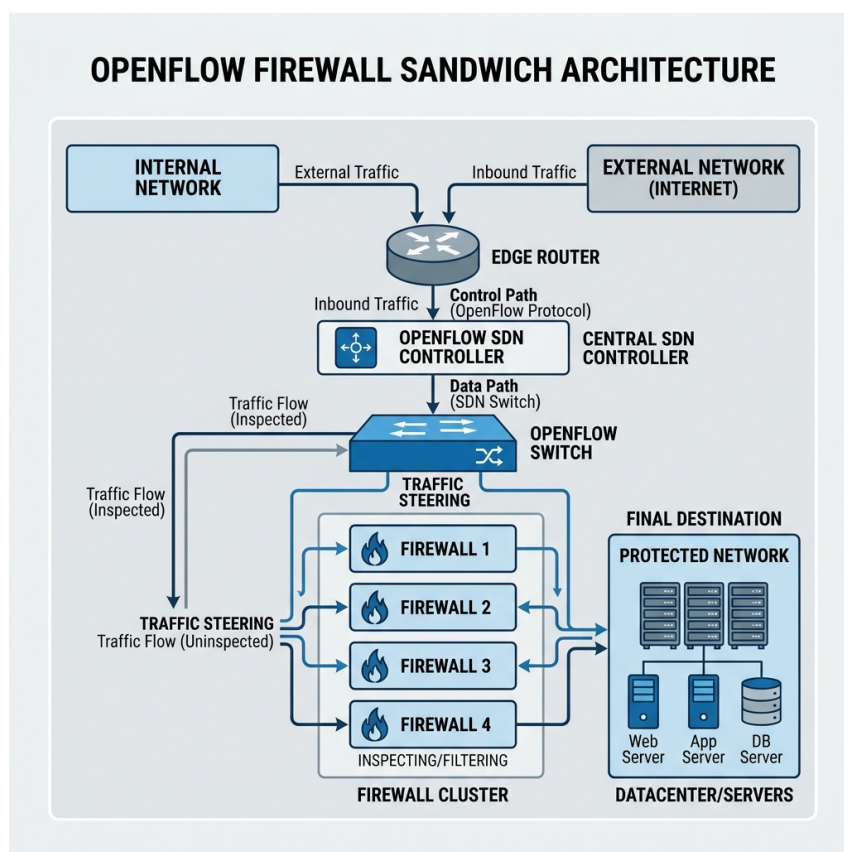


Figura 2.6: Architettura Firewall Sandwich governata tramite SDN e OpenFlow.

2.3 Topologie di Rete: Dal 3-Tier allo Spine-and-Leaf

Storicamente, i datacenter adottavano una topologia **3-tier** (Access, Distribution, Core), ottimizzata per il traffico **Nord-Sud** (client-server). I collegamenti ridondanti creavano inevitabilmente loop fisici. Se un pacchetto broadcast iniziava a ciclare, innescava un catastrofico **Broadcast Storm** in grado di paralizzare l'intero datacenter in pochi secondi. Per evitarlo si utilizzava il protocollo **STP (Spanning Tree Protocol)**, che disabilitava preventivamente i link fisici ridondanti. I difetti dell'STP erano molteplici: enormi sprechi di banda (fino al 50% dei cavi posati rimaneva dormiente) e lenti tempi di convergenza in caso di guasto (i ricalcoli della topologia bloccavano momentaneamente il traffico).

Oggi, l'esplosione dei microservizi, del cloud e dell'IA ha reso assolutamente dominante il traffico **Est-Ovest** (server-to-server). La topologia standard dei moderni data center è diventata la **Spine-and-Leaf** (derivata dalle reti di *Clos*):

SPINE-LEAF NETWORK ARCHITECTURE

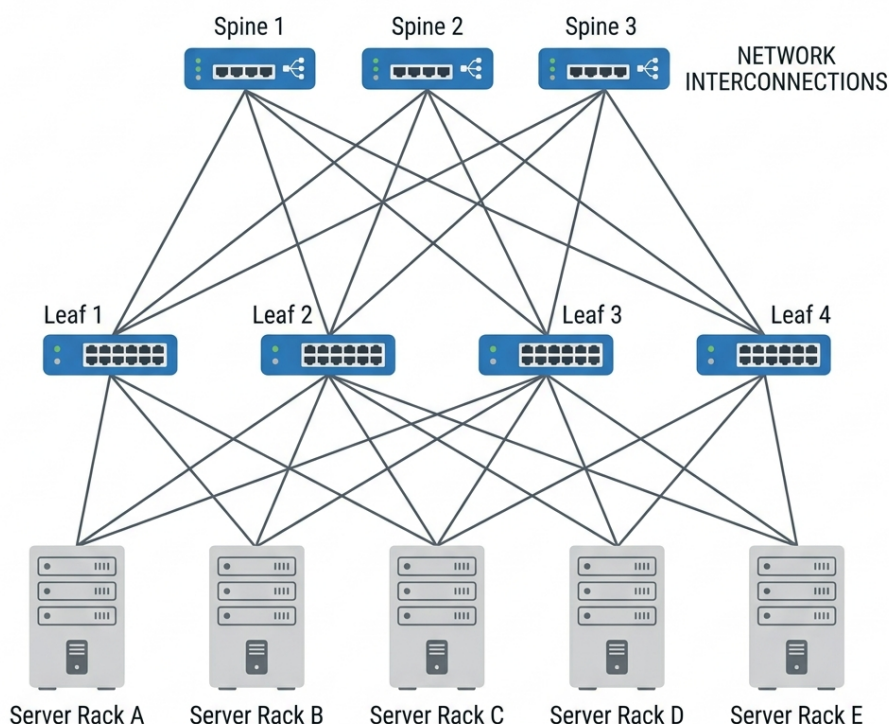


Figura 2.7: Architettura Spine-Leaf: connessione Full-Mesh tra livello Leaf e livello Spine.

L'architettura si sviluppa su due livelli fisici rigorosi:

- **Leaf Layer (Accesso):** Costituito dagli switch ToR (Top-of-Rack). I server fisici si collegano *solo* a questi switch. A loro volta, gli switch Leaf **non sono mai collegati tra loro**.
- **Spine Layer (Backbone):** Il livello centrale di aggregazione. Ogni singolo switch Leaf possiede almeno un collegamento uplink in fibra verso **ogni** singolo switch Spine (creando una configurazione *Full-Mesh* tra i due livelli). Anche gli switch Spine **non si parlano mai direttamente tra di loro**.

Vantaggi e Oversubscription

- **Latenza Deterministica:** Ogni server comunica con un qualsiasi altro server impiegando esattamente lo stesso numero di salti (1 hop logico: *leaf* → *spine* → *leaf*).
- **ECMP (Equal-Cost Multi-Path):** Senza i tradizionali loop fisici, l'STP viene sostituito dal routing L3. Il traffico viene bilanciato uniformemente (load balancing) su *tutti* gli uplink disponibili. Se uno switch Spine fallisce, la rete si degrada solo frazionalmente in banda, ma non si interrompe.
- **Oversubscription Ratio:** Il rapporto tra banda totale di downlink (verso i server) e uplink (verso gli Spine). Un tipico cloud usa rapporti 3:1. Gli ambienti HPC o AI esigono un rapporto **1:1** (detto *Non-blocking*), garantendo che tutti i server possano trasmettere al massimo contemporaneamente senza colli di bottiglia.

2.4 Segmentazione e Virtualizzazione della Rete

2.4.1 Jumbo Frames e MTU

Prima di frammentare logicamente la rete, è fondamentale definirne il payload massimo. L'MTU (**Maximum Transmission Unit**) standard in Ethernet è di 1500 byte. Tuttavia, in ambienti datacenter (specialmente per il traffico storage), segmentare file enormi in blocchi da 1.5 KB causa un overhead insostenibile di CPU per elaborare milioni di header. Si abilitano quindi i **Jumbo Frames**, portando l'MTU a **9000 byte (9 KB)**. La trappola operativa è la frammentazione: l'MTU deve essere configurato in modo rigorosamente identico su ogni singolo switch e server lungo il percorso, pena la caduta drastica delle performance o la perdita silenziosa dei pacchetti (*black holing*).

2.4.2 VLAN (Virtual LAN)

L'header Ethernet classico contiene un indirizzo MAC di destinazione (6 byte), un MAC sorgente (6 byte) e un campo EtherType (2 byte). Le VLAN (standard IEEE 802.1Q) permettono di suddividere una rete fisica in più domini di broadcast logici, inserendo un tag di 4 byte subito dopo il MAC sorgente. Questo tag 802.1Q include un campo TPID (che indica la presenza della VLAN) e un TCI, il quale a sua volta contiene 3 bit per la priorità (QoS) e **12 bit per l'identificativo VLAN (VID)**. Essendo l'ID VLAN lungo esattamente 12 bit, il limite teorico invalicabile è di **4094 reti** utilizzabili ($2^{12} - 2$), un numero assolutamente insufficiente per i moderni cloud provider multi-tenant che devono ospitare centinaia di migliaia di reti isolate.

2.4.3 VXLAN e Overlay

Per superare la rigidità architetturale e il limite di 4096 reti delle VLAN tradizionali, il datacenter cloud moderno usa **VXLAN (Virtual eXtensible LAN)**. È un protocollo di *overlay* (Mac-in-UDP) che incapsula interi pacchetti Layer 2 (Ethernet) all'interno di pacchetti di trasporto Layer 3 (UDP/IP). Questo innalza il limite a **16 milioni di reti**

virtuali isolate (grazie a un identificativo VNI a 24 bit) e permette di creare domini di broadcast Layer 2 arbitrari su un'infrastruttura fisica di routing Layer 3. Ciò significa che due macchine virtuali possono credere di essere collegate allo stesso switch fisico (stessa subnet L2), anche se si trovano in due rack fisicamente distanti instradati in L3.

2.5 Fabric ad Alte Prestazioni (HPC e AI)

In contesti HPC e di addestramento AI, l'overhead software diventa il nemico primario. Come visto, un frame attraversa l'hardware dello switch (il cut-through ASIC) in circa **600 ns**. Tuttavia, solo per far attraversare al dato lo stack software del protocollo TCP/IP all'interno del sistema operativo del server (gestione interrupt, allocazione buffer), si introducono circa **70 μ s** di latenza (oltre 100 volte il tempo di switching), un valore che l'inventore del TCP Vint Cerf aveva associato al problema del *bufferbloat* (l'accumulo distruttivo di pacchetti nei buffer intermedi).

In questi scenari domina **InfiniBand**, una rete che offre latenze di 1-3 μ s. Il segreto è l'implementazione del protocollo **RDMA (Remote Direct Memory Access)**, che permette a un server di scrivere direttamente nella memoria RAM di un altro server scavalcando completamente l'intervento della CPU e del sistema operativo ricevente. Oggi, RDMA è disponibile anche su reti Ethernet avanzate tramite il protocollo **RoCE** (RDMA over Converged Ethernet).

Full Fat Tree

Per evitare l'*overbooking* (sovraccarico degli uplink) critico per l'HPC, si usa la topologia **Full Fat Tree**: una struttura non-bloccante in cui la larghezza di banda verso i livelli superiori è esattamente pari a quella dei livelli inferiori, garantendo comunicazioni senza strozzature.

Capitolo 3

Storage: Architetture e Servizi

3.1 Evoluzione del Mezzo di Memorizzazione

Lo storage rappresenta il secondo pilastro fondamentale di un datacenter, il luogo dove i dati persistono alla perdita di alimentazione (persistenza). La regola assoluta dello storage è una sola: **perdere i dati è il peccato imperdonabile**.

3.1.1 Dal Nastro Magnetico all'HDD e all'SSD NVMe

Storicamente, i primi datacenter utilizzavano **Nastri Magnetici (Tape)**: cartucce robotiche lente ma dalla capacità e densità enormi. Il nastro costringe ad un accesso sequenziale ai dati (non si può saltare alla fine senza "srotolare" tutto), ma è ancora oggi impiegato per il cold storage e i backup offline impenetrabili ai ransomware (Air-gap).



Figura 3.1: Interno di un Hard Disk Drive magnetico meccanico tradizionale.

Successivamente, si è passati agli **HDD (Hard Disk Drive)**, che offrono accesso casuale ai dati. Tuttavia, essi sono limitati da vincoli meccanici (rotazione dei piatti, tempo di *seek* della testina), con latenze invalicabili di circa 3-5 ms. Le configurazioni storiche per i server usavano interfacce complesse in cascata (es. **SCSI**), oggi sostituite da protocolli SAS o SATA. Attualmente gli HDD sono relegati quasi esclusivamente all'archiviazione di massa a basso costo.

La transizione verso gli **SSD (Solid State Drive)** ha abbattuto la latenza a poche centinaia di microsecondi. Tuttavia, l'uso dei vecchi controller SATA si è rivelato rapidamente un collo di bottiglia. La rivoluzione successiva è stata l'introduzione dello standard **NVMe (Non-Volatile Memory Express)**, un protocollo che collega direttamente la

memoria flash al bus **PCIe**, portando la latenza a $\sim 5\text{-}20\ \mu\text{s}$ e il throughput a decine di GB/s.

Gerarchia della Memoria Moderna

Con NVMe, il divario prestazionale tra DRAM (nanosecondi) e storage permanente (microsecondi) si è ridotto da 6 a soli 2 ordini di grandezza, permettendo architetture software impensabili prima, come database (es. RocksDB) che operano direttamente su storage senza pesanti strati di caching in RAM.

3.1.2 Celle NAND e Asimmetria

A differenza della DRAM, la memoria NAND Flash presenta un'asimmetria intrinseca: le operazioni di lettura sono veloci, mentre le scritture richiedono un ciclo PE (Program/Erase) molto più lento. La densità dei bit per cella determina il tipo di SSD:

- **SLC (Single-Level Cell):** 1 bit, altissima resistenza (100k cicli PE), altissimo costo.
- **MLC, TLC, QLC:** rispettivamente 2, 3 e 4 bit per cella. Costi decrescenti ma resistenza (endurance) e prestazioni in scrittura in rapido calo.

Ritenzione dei Dati e Over-provisioning

Due problemi ingegneristici critici affliggono gli SSD moderni:

1. **Longevità (Data Retention):** Per prevenire la dissipazione naturale della carica elettronica intrappolata nelle celle, il controller dell'SSD è costretto a leggere e riscrivere "silenziosamente" in background i propri blocchi NAND ogni ~ 2 settimane. Senza alimentazione prolungata (es. nel Cold Storage), i dati su SSD evaporano.
2. **Over-provisioning:** Un SSD non può sovrascrivere direttamente un blocco contenente dati parziali; deve leggere, cancellare interamente il blocco e riscrivere (Write Amplification). Se un SSD supera il 90% di riempimento, lo spazio libero per queste manovre (*Garbage Collection*) si esaurisce, causando un crollo drastico e subitaneo delle performance. Per evitarlo, si ricorre al pre-conditioning e all'Over-provisioning hardware (allocando in fabbrica un 10-20% di spazio flash aggiuntivo invisibile al sistema operativo).

3.1.3 Misure di Performance: IOPS, Latenza e Bandwidth

Latenza e Bandwidth (throughput) da sole non sono sufficienti per caratterizzare le prestazioni di uno storage, poiché dipendono fortemente dal pattern di accesso (sequenziale o casuale). Una metrica fondamentale è l'**IOPS** (Input/Output Operations Per Second), ovvero il numero di operazioni di I/O completate in un secondo.

- **Analogia del supermercato:** Gli IOPS rappresentano i clienti serviti al secondo, la latenza è il tempo impiegato in cassa per ogni cliente, e il parallelismo (le code) sono le casse aperte.

- La relazione base per una singola coda è: $IOPS \approx \frac{1}{\text{latenza}}$.
- Grazie alla natura asincrona dello storage, protocolli come NVMe (che supportano fino a 64.000 code parallele) permettono di scalare immensamente il throughput elaborando molte richieste simultaneamente.

3.2 Aggregazione e Protezione: RAID e Resilienza

Un singolo disco è un single point of failure inaccettabile. Si utilizza quindi il **RAID** (Redundant Array of Independent Disks):

- **RAID 0 (Striping)**: Nessuna ridondanza, altissime prestazioni, rischio massimo.
- **RAID 1 (Mirroring)**: Copia identica (Mirrored LUN), perde il 50% dello spazio ma garantisce disponibilità istantanea in caso di guasto.
- **RAID 5**: Ridondanza calcolata tramite funzione logica **XOR**. Tolleranza alla rottura di 1 solo disco. Se un secondo disco si rompe durante la delicata fase di *rebuild* (che può durare giorni sui moderni dischi capienti), si perdono tutti i dati.
- **RAID 6**: Usa doppia parità, permettendo il fallimento simultaneo di **2 dischi**.

Il Tramonto del RAID Hardware e l'Erasure Coding

L'avvento degli SSD NVMe (con throughput di decine di GB/s) ha reso paradossalmente obsoleti i costosi *Controller RAID Hardware*. Il chip del controller non riesce a star dietro matematicamente alla velocità di calcolo della parità necessaria per dischi così veloci, diventando il nuovo collo di bottiglia. Oggi domina il **Software RAID** (es. `mdadm` su Linux o file system come ZFS), che delega il calcolo delle parità alle potentissime CPU host multicore.

Nei sistemi cloud distribuiti su vasta scala (come l'Object Storage), il RAID classico fallisce. Viene sostituito dall'**Erasure Coding ($n=m+k$)**: un file viene spezzato in m frammenti dati, ai quali si aggiungono k frammenti di parità calcolati matematicamente. I frammenti vengono dispersi su decine di server in rack diversi. È possibile ricostruire il file intero avendo a disposizione *qualsiasi* m frammenti. Questo offre maggiore resilienza del RAID a un costo di archiviazione inferiore, ma con un overhead di calcolo CPU maggiore.

3.3 Architetture di Storage nel Datacenter

3.3.1 SAN (Storage Area Network)

La SAN centralizza lo storage separandolo dai server, comunicando a livello di blocco (espone dischi grezzi, non file). I protocolli dominanti per questo fabric sono:

- **Fibre Channel (FC)**: Standard legacy predominante prima dell'era NVMe, si basa sul protocollo **SCSI** per la movimentazione dei blocchi. È una rete fisicamente e logicamente separata dall'Ethernet standard, spesso costruita con topologia a doppio fabric fisicamente ridondato (Fabric A e Fabric B). Richiede schede dedicate sui server, dette **HBA (Host Bus Adapter)**. Identifica i nodi non tramite MAC address, ma tramite indirizzi a 64 bit chiamati **WWN (World Wide Name)**.

- **iSCSI:** Alternativa economica che trasporta i comandi SCSI all'interno di normali pacchetti TCP/IP su rete Ethernet. Funziona tramite un'architettura client-server: il server host agisce da **iSCSI Initiator** (che richiede i blocchi), mentre lo storage array è l'**iSCSI Target** (che li fornisce). I nodi sono identificati tramite stringhe IQN.
- **FCoE (Fibre Channel over Ethernet):** Il tentativo storico di unificare le due reti. Incapsula i frame FC (SCSI) direttamente dentro l'Ethernet standard (Lossless Ethernet usando protocolli DCB/PFC), eliminando la necessità di cablaggio separato, senza l'overhead del TCP/IP. Oggi superato quasi totalmente da **NVMe-over-Fabrics (NVMe-oF)**.

Sicurezza SAN: Zoning e Masking

Per evitare che un server formatti per errore il disco (o intercetti i dati) di un altro server, si usano due barriere:

1. **Zoning (Lato Rete):** Applicato sugli switch, crea zone hardware chiuse. Simile al concetto di VLAN nell'Ethernet, impedisce a due HBA che non sono nella stessa zona di comunicare o "vedersi" a vicenda.
2. **LUN Masking (Lato Storage):** Lo storage array autorizza l'accesso a un disco logico (**LUN, Logical Unit Number**) esclusivamente allo specifico indirizzo WWN (per FC) o IQN (per iSCSI) del server proprietario.

3.3.2 NAS (Network Attached Storage)

Il NAS opera a un livello di astrazione più alto, connesso tramite rete Ethernet standard (protocolli NFS o SMB).

- **Astrazione:** Espone un File System logico (*file e directory*).
- **Limiti e mmap:** Maggiore latenza di rete. In particolare, il NAS distrugge le performance applicative in caso di uso massivo della direttiva **memory mapping (mmap)** per accesso casuale (es. database o file virtual disk VM). Il *mmap* mappa un file logico del disco direttamente nello spazio RAM di un processo (Page Cache). Se il file risiede in remoto su un NAS, ogni singolo *page fault* (lettura di un frammento mancante in RAM) scatena sincronicamente un intero bloccante round-trip di rete Ethernet, mascherando un disastroso ritardo di I/O dietro una semplice istruzione di memoria, portando a stalli (*stall*) applicativi. Questo problema non si verifica con lo storage a blocchi SAN (DMA diretto).
- **Casi d'uso:** File sharing, backup, documenti aziendali.

3.3.3 Object Storage

Sviluppato per gestire scalabilità a livello di Exabyte (es. Amazon S3). Abbandona cartelle e blocchi, utilizzando **oggetti** piatti allocati in *bucket* e interrogabili tramite API REST. Per garantire la scalabilità e distribuire i dati senza un collo di bottiglia centrale, utilizza algoritmi di routing come il **Consistent Hashing**. Ideale per dati statici e massivi, come dataset di Machine Learning o archivi medici.

3.3.4 HCI (Hyper-Converged Infrastructure)

Per decenni il mercato datacenter è stato dominato da grandi vendor che fornivano SAN e NAS hardware proprietari (es. i famosi array **EMC Symmetrix** o i **Filer NetApp**), apparati costosissimi e dal difficile scaling orizzontale. Per abbattere i costi e scalare linearmente, si è passati all'**HCI**, che trasforma lo storage SAN centralizzato in un approccio distribuito (*Scale-out*). Ogni server (nodo commodity x86) contiene CPU, RAM e dischi NVMe locali. Un livello software (Software-Defined Storage) raggruppa tutti i dischi locali di tutti i nodi e li presenta all'hypervisor come un'unica grande SAN virtuale condivisa.

Vantaggio dell'HCI

Data Locality: Il software tenta sempre di scrivere/leggere i dati di una VM nel disco fisicamente residente nello stesso server che la esegue, azzerando il traffico di rete Nord-Sud e sfruttando al massimo la banda PCIe locale.

3.3.5 Storage per l'AI: Vector Database

L'emergere dell'Intelligenza Artificiale generativa ha richiesto nuovi paradigmi di memorizzazione. La ricerca testuale classica era puramente *sintattica* (come nel modello *bag of words*), mentre i modelli neurali operano sulla *semantica*.

- **Embeddings:** Le reti neurali (LLM) trasformano testi, immagini o audio in vettori densi ad altissima dimensionalità (es. 3000 dimensioni). Questi vettori, chiamati *embedding*, rappresentano il reale significato del dato in base al contesto (es. distinguono "Python" come linguaggio da "Python" come serpente).
- **Vector Database:** Sono una nuova generazione di database NoSQL progettati specificamente per immagazzinare e indicizzare questi enormi vettori. Per rispondere a una query, vettorializzano la domanda dell'utente e cercano i documenti semantici più simili calcolando la **distanza coseno** nello spazio vettoriale.

RAG: Retrieval-Augmented Generation

I Vector Database sono il motore dell'architettura **RAG**, tecnica essenziale per limitare le "allucinazioni" delle AI. Prima di rispondere a un prompt, il sistema interroga il vector database, recupera i documenti aziendali realmente pertinenti, li inietta nel prompt e obbliga il LLM a formulare la risposta basandosi unicamente su quei fatti verificati.

3.4 Servizi Avanzati e Resilienza

3.4.1 Efficienza: Deduplicazione, Compressione e Tiering

Oltre a conservare blocchi, i controller storage moderni (NAS/SAN) offrono servizi a valore aggiunto (spesso *in-line*, calcolati in tempo reale durante la scrittura tramite chip ASIC dedicati):

- **Deduplicazione:** Identifica e rimuove blocchi di dati fisicamente identici (es. se ci sono 100 macchine virtuali Windows con lo stesso file di sistema operativo, il dato viene salvato una volta sola sul disco). Un contatore referenziale logico tiene traccia delle copie per gli host.
- **Compressione:** Riduzione matematica dell'impronta dei dati sul disco.
- **Caching e Auto-Tiering:** Promozione automatica dei blocchi di dati più letti/-scritti ("hot data") da dischi lenti a tier molto veloci (es. RAM NVRAM o SSD NVMe) in modo del tutto trasparente all'applicazione.
- **Multipath I/O:** A livello server, ogni HBA viene collegato a fabric di rete paralleli (es. switch A e B) per avere percorsi logici simultanei verso la medesima LUN dello storage array, garantendo load-balancing e failover istantaneo (MPIO).

3.4.2 Snapshot, Thin Provisioning e Replica

- **Thin Provisioning:** Permette di allocare logicamente agli utenti più spazio di quanto fisicamente presente nello array, basandosi sul principio che lo spazio allocato viene riempito raramente (*Overbooking*). Il rischio fatale è esaurire fisicamente lo spazio libero se tutti scrivono, causando un blocco panico sistemico di tutto il datacenter (nessuno può più scrivere su disco).
- **Snapshot vs Clone:** La *Snapshot* è una fotografia istantanea del volume a costo zero in termini di spazio. Sfrutta il meccanismo **Copy-on-Write (COW)**: congela i blocchi originali e alloca nuovi blocchi (file differenziali) solo per le successive modifiche. **Impatto sulle performance:** un'alta catena di snapshot attive degrada fortemente le prestazioni I/O in lettura. Un **Clone** è invece una copia fisica completa bit-a-bit (comporta consumo di spazio e tempo).
- **Crash-consistent vs Application-consistent:** Una snapshot di base è sempre *crash-consistent* (equivalente allo staccare brutalmente la spina, senza flush delle cache in RAM), il che corrompe inesorabilmente i database. Per prevenirlo, l'infrastruttura utilizza agenti come **Windows VSS (Volume Shadow Copy Service)** che si coordinano con il SO per "congelare" un attimo l'I/O applicativo (*quiesce*), scattare la snapshot e riprendere in modo *application-consistent*.
- **Replica Sincrona/Asincrona:** Indispensabile per il Disaster Recovery. Nel caso di siti geografici vicini (entro 15ms latenza), si usano **Metro Cluster** con replica sincrona active-active.

3.4.3 Backup vs Replica

La **replica** è una copia speculare in tempo reale: protegge dai guasti hardware, ma propaga istantaneamente cancellazioni umane o attacchi ransomware. Il **backup** è un salvataggio point-in-time e sconnesso (offline) che permette il recupero.

Nel dimensionamento di un sistema di continuità aziendale due metriche sono fondamentali:

- **RPO (Recovery Point Objective):** Quantità di dati che l'azienda è disposta a perdere (frequenza dei backup).

- **RTO (Recovery Time Objective):** Tempo massimo tollerabile per il ripristino dell'infrastruttura.

GDPR e Data Breach

Secondo il GDPR, un *data breach* non è solo il furto dei dati (perdita di *confidenzialità*), ma include anche la perdita di *integrità* (dati corrotti) e di **disponibilità** (dati integri ma non accessibili, ad esempio durante un attacco ransomware che cifra l'unico storage primario).

Capitolo 4

Compute e Virtualizzazione

4.1 Architettura Compute: CPU, GPU e NPU

L'elaborazione dei dati (Compute) è il terzo pilastro del datacenter. L'evoluzione hardware moderna è guidata dalla fine della scalabilità lineare del singolo core (Legge di Moore) e dalla necessità di efficienza e specializzazione per carichi di lavoro legati all'Intelligenza Artificiale.

4.1.1 Sostenibilità Energetica

I datacenter consumano enormi quantità di energia elettrica e acqua. Le metriche di riferimento sono:

- **PUE (Power Usage Effectiveness):** Rapporto tra energia totale consumata dal datacenter e quella effettivamente usata dai server. Il valore ideale è 1.0.
- **WUE (Water Usage Effectiveness):** Consumo idrico per kWh, critico nei datacenter che usano raffreddamento evaporativo (Torri di raffreddamento).

4.1.2 Architettura CPU: Pipeline e Multi-Core

Storicamente, per aumentare le prestazioni, si aumentava la frequenza di clock e si ottimizzava la **Pipeline delle istruzioni** (suddivisa classicamente nelle fasi: *Instruction Fetch, Decode, Execute, Memory Access, Write-back*). Aumentando la profondità della pipeline (fino a 20+ stadi), si potevano eseguire più istruzioni sovrapposte nel tempo, ma gli errori di previsione dei salti (*Branch Misprediction*) causavano lo stallo e lo svuotamento completo della coda ("bolle" nella pipeline), vanificando l'efficienza.

Arrivati al limite fisico del calore dissipabile (Power Wall), si è passati al parallelismo Multi-Core. Per superare i colli di bottiglia della condivisione della RAM, le CPU moderne adottano architetture **NUMA (Non-Uniform Memory Access)**, in cui ogni processore ha un proprio banco di memoria locale ad accesso diretto rapido; l'accesso alla RAM degli altri processori avviene tramite interconnessioni esterne a latenza maggiore.

Hyperthreading e SMT (Simultaneous Multithreading)

Per ridurre al minimo i tempi morti (idle time) in cui un core attende dati dalla memoria, i progettisti hanno introdotto l'**Hyperthreading** (o SMT). Il trucco consiste nel duplicare solo le risorse "leggere" (come i registri e il program counter), mantenendo condivise le costose unità di calcolo fisico (ALU). Il sistema operativo vede **due core logici** per ogni

core fisico: quando un thread è in stallo per un *cache miss*, il core scambia istantaneamente contesto ed esegue le istruzioni dell'altro thread. Sebbene eccellente per server web e database ad alto I/O, l'HT è **dannoso in ambito HPC** (High Performance Computing): se due thread devono fare calcolo intensivo matematico simultaneo, lotteranno aspramente per l'unica ALU condivisa, riducendo le performance generali. Nei supercomputer l'HT viene perciò disabilitato.

Overbooking e Memory Ballooning

La virtualizzazione si fonda sull'**Overbooking** (sovra-allocazione logica delle risorse fisiche sottostanti). Oltre all'ovvio over-commit delle vCPU, si esegue overbooking pesante sulla RAM. Un meccanismo chiave è il **Memory Ballooning**: l'hypervisor inietta un driver-palloncino nel kernel del SO ospite. Se il server fisico sta esaurendo la vera RAM, l'hypervisor "gonfia" il palloncino. Il SO ospite crede di aver bisogno urgentemente di memoria interna, swappa i propri processi meno usati sul suo disco virtuale e restituisce la RAM vera al palloncino (e quindi all'hypervisor). L'hypervisor maschera inoltre enormi quantità di RAM tramite *Memory Page Sharing* (es. KSM deduplica le pagine di sistema operativo identiche tra decine di VM Windows uguali).

Vulnerabilità Side-Channel

Condividere fisicamente la cache L1 e i buffer di esecuzione tra due thread apre la porta ad attacchi speculativi come **Spectre** o *MDS*. Un thread malevolo può dedurre dati segreti del thread "fratello" misurando la latenza della cache. Per massima sicurezza, alcuni cloud provider disabilitano del tutto l'SMT.

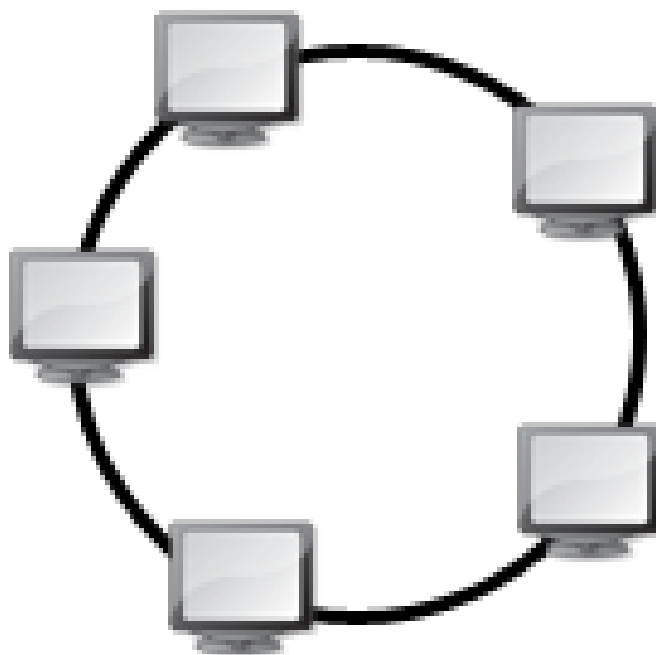


Figura 4.1: Schema logico di una topologia Ring, storicamente usata per l'interconnessione interna dei core CPU.

Interconnessione interna: Crossbar vs Bus

A livello di interconnessione interna (die), le prime CPU multi-core usavano un *Bus Condiviso*, che diventava subito un collo di bottiglia: solo un core alla volta poteva parlare. Oggi si utilizza la **Crossbar** (o una *Mesh Network*): una matrice di interconnessione che permette a molteplici core di comunicare simultaneamente con banchi di memoria o I/O diversi senza intralciarsi, garantendo *bisection bandwidth* completa.

Questo approccio reticolare si sposa perfettamente con l'architettura a **Tile (o Chiplet)**: i processori moderni non sono più un unico monolite di silicio prone a difetti, ma un mosaico di piccoli chip iperspecializzati (Compute Tile, I/O Tile) stampati con nodi differenti e incollati sullo stesso package, migliorando enormemente le rese produttive (yield) e abbassando i costi. Esempi concreti di questo approccio sono il fabric di interconnessione **AMD Infinity Fabric** per unire i chiplet nei processori EPYC, e le tecnologie di packaging avanzato 3D come **Intel EMIB e Foveros** (usate nelle architetture Meteor Lake e Sapphire Rapids) che impilano i die di silicio verticalmente.

4.1.3 GPU e NPU: Il cambio di paradigma

- **CPU:** Ottimizzata per la *minima latenza*. Pochi core molto complessi con esecuzione *Out-of-Order* e grandi cache.
- **GPU:** Ottimizzata per il *massimo throughput*. Migliaia di core semplici che eseguono lo stesso calcolo in parallelo (SIMT), ideale per l'addestramento dell'AI.
- **NPU (Neural Processing Unit):** Circuiti dedicati e specializzati esclusivamente nel calcolo tensoriale e inferenza AI a bassa precisione numerica e bassissimo consumo energetico.

4.2 Architetture Server Fisici e BMC

L'involucro fisico (form factor) dei server si è diversificato negli anni per rispondere a diverse esigenze di densità, potenza di calcolo e dissipazione termica.

4.2.1 Server Rack (1U / 2U)



Figura 4.2: Server Rack 1U standard con alloggiamenti per dischi frontali (hot-swap).

È il formato "general-purpose" più diffuso nei data center tradizionali. Un server rack è un computer completo e indipendente (dotato di propri alimentatori, ventole, CPU, RAM e dischi) racchiuso in uno chassis metallico standardizzato largo 19 pollici. L'altezza si misura in unità rack (U, circa 4,4 cm). Sono ideali per workload eterogenei, ma in grandi quantità richiedono un enorme lavoro di cablaggio singolo per ogni macchina (cavi di alimentazione e di rete separati per ogni singolo server).

4.2.2 Blade Server



Figura 4.3: Chassis Blade System: decine di server-lama inseriti in un singolo modulo.

Progettati per ottenere l'altissima densità necessaria nei cluster di virtualizzazione. Un sistema Blade si divide in due componenti: lo **Chassis** (l'involucro esterno che fornisce alimentazione condivisa, mega-ventole e switch di rete integrati) e le **Lame** (i server veri e propri). La lama (Blade) è sottilissima perché contiene solo CPU, RAM e pochi dischi; non ha ventole o alimentatori propri. Inserendola nello chassis, si collega automaticamente al *midplane* posteriore, azzerando quasi totalmente il disordine del cablaggio nel rack.

4.2.3 Server HGX (AI GPU Server)



Figura 4.4: Server HGX (aperto): GPU ad alte prestazioni coperte da enormi dissipatori metallici per gestire il calore estremo.

Con l'esplosione dell'intelligenza artificiale generativa, i normali server rack non sono più sufficienti. Un server HGX è una macchina immensa (spesso 4U o 8U, pesante oltre 100 kg) progettata esclusivamente attorno alle GPU (es. NVIDIA H100). Le GPU non sono più inserite come schede PCIe estraibili, ma sono saldate direttamente su una *baseboard* proprietaria e interconnesse tramite bus dedicati (come NVLink) che scavalcano il collo di bottiglia del PCIe. Generano un calore spaventoso, rendendo spesso obbligatorio il passaggio al raffreddamento a liquido (*Direct-to-Chip Liquid Cooling*).

4.2.4 Standardizzazione Hyperscaler: Open Compute Project (OCP)

Creato originariamente da Meta (Facebook) nel 2011, l'OCP (**Open Compute Project**) è un consorzio no-profit che pubblica le specifiche hardware open source per server e rack da datacenter. L'idea nasce da un'esigenza economica: i server tradizionali di brand contengono molte funzionalità estetiche o di gestione (frontalini in plastica, display, alimentatori ridondanti per singolo server) inutili su scala hyperscaler. Le specifiche OCP definiscono macchine essenziali (spesso prodotte direttamente da costruttori ODM taiwanesi come Quanta o Foxconn), permettendo risparmi enormi. Un'innovazione chiave dell'OCP è il design del rack: l'alimentazione non viene convertita in ogni singolo server,

ma centralizzata a monte nel rack e distribuita ai server tramite busbar a 12V o 48V, aumentando l'efficienza energetica e la densità.

BMC (Baseboard Management Controller) e lo Standard Redfish

Tutti i server di livello enterprise montano un **BMC** (es. iLO di HPE o iDRAC di Dell). Si tratta di un micro-computer indipendente inserito nella scheda madre, dotato di una porta di rete dedicata (*out-of-band management*). Essendo alimentato separatamente, permette il controllo fisico del server (accensione, configurazione BIOS, console KVM virtuale) anche se il sistema operativo principale è bloccato o se il server è fisicamente spento.

Redfish: L'API del Datacenter Moderno

Storicamente, i BMC venivano gestiti tramite il protocollo IPMI, che col tempo si è rivelato insicuro e difficile da integrare. Per questo motivo, il consorzio DMTF ha introdotto **Redfish**, lo standard moderno per la gestione dell'infrastruttura.

- **Come funziona:** Redfish espone tutti i parametri hardware tramite una **RESTful API** standard. Utilizza i protocolli del web moderno (HTTPS per la sicurezza) e restituisce i dati in formato **JSON** (OData), facilmente leggibile sia dagli umani che dagli script.
- **Cosa fa:** Permette di automatizzare su larga scala le operazioni più tediose. Tramite semplici script Python o strumenti come Ansible, si possono inviare richieste web (GET, POST, PATCH) per monitorare le temperature, aggiornare i firmware, modificare l'ordine di boot o riavviare un intero rack di server, indipendentemente dalla marca dell'hardware sottostante.

4.3 Virtualizzazione e Live Migration

La virtualizzazione svincola il software dall'hardware fisico, permettendo a più Sistemi Operativi (Guest) di girare simultaneamente sullo stesso server fisico (Host). Il cuore di questo processo è l'**Hypervisor** (o Virtual Machine Monitor).

4.3.1 L'Evoluzione dell'Hypervisor: Dai Ring CPU alla Virtualizzazione Hardware

L'architettura x86 originale non era progettata per essere virtualizzata. Prevedeva quattro livelli di privilegio logici chiamati **Ring**, da Ring 0 (massimo privilegio, destinato al kernel del SO) a Ring 3 (spazio utente). In un ambiente virtualizzato, l'Hypervisor deve avere il controllo assoluto, quindi deve risiedere in Ring 0. Questo costringeva i sistemi operativi Guest a scalare in Ring 1 (modalità non privilegiata, definita *Ring De-privileging*). Poiché i SO Guest non sapevano di essere virtualizzati, tentavano di eseguire istruzioni privilegiate (es. manipolazione hardware) che, fallendo, venivano intrappolate dall'Hypervisor. Quest'ultimo doveva tradurle ed emularle in tempo reale (tecnica della **Binary Translation**), causando un enorme e insostenibile overhead prestazionale.

Il problema è stato risolto nel 2005 dai produttori di CPU con l'introduzione di estensioni hardware dedicate (**Intel VT-x** e **AMD-V**). Queste istruzioni hanno creato un nuovo livello di privilegio inferiore allo zero (spesso chiamato colloquialmente **Ring -1**).

Oggi l'Hypervisor risiede in Ring -1, permettendo ai kernel dei sistemi operativi Guest di operare nel loro nativo Ring 0, eseguendo istruzioni in modo diretto e a velocità hardware, annullando la necessità di lenta traduzione binaria.

4.3.2 Astrazione dell'Hardware: vCPU, Pause e Resume

La macchina virtuale non vede la reale CPU fisica, ma una **vCPU** (Virtual CPU) configurata dall'hypervisor. Questo è un dettaglio cruciale per la portabilità: l'hypervisor può nascondere set di istruzioni troppo recenti (es. AVX-512) mascherando le reali capacità del processore fisico, in modo da poter migrare liberamente la VM anche verso un server più vecchio che non possiede quelle istruzioni.

Inoltre, siccome l'intero ambiente Guest è solo software in esecuzione su un ciclo *fetch-execute* controllato, l'hypervisor gode del potere di **Pause and Resume**. Può congelare istantaneamente il sistema operativo salvando *lo stato completo della VM* (registri della CPU, stato della memoria e disco virtuale). Il SO virtualizzato non si accorgerà di nulla, per lui il tempo si sarà semplicemente fermato. Tramite gli **Integration Services**, l'hypervisor forza poi la sincronizzazione del clock alla ripresa (vitale per protocolli come Kerberos che falliscono se l'orario si disallinea), invia segnali di *heartbeat* per capire se il kernel Guest è crashato, e orchestra spegnimenti puliti (*graceful shutdown*).

Windows Dual Boot e Secure World con WSL2

Oggi la virtualizzazione è pervasiva anche sui dispositivi client. All'avvio, un PC con Windows moderno (VBS abilitato) non fa un normale avvio nudo, ma lancia un hypervisor (Hyper-V) che istanzia due ambienti. Il primo è un micro-OS nascosto (**Secure World**) che isola l'hardware crittografico (es. chiavi BitLocker). Subito dopo viene lanciato l'ambiente Windows desktop principale, che gira quindi *virtualizzato*. Anche se un hacker ottiene i privilegi massimi (Ring 0) sul kernel di Windows, non potrà mai rubare le chiavi crittografiche del Secure World. Lo stesso hypervisor client trasparente rende possibile **WSL2** (Windows Subsystem for Linux), che esegue un vero kernel Linux completo in una micro-VM ultraleggera parallelamente a Windows.

4.3.3 Il Virtual Switch (vSwitch)

Per connettere le VM al mondo esterno, l'Hypervisor implementa un **Virtual Switch** in memoria RAM. Le macchine virtuali possiedono schede di rete logiche (**vNIC**), che si collegano alle porte del vSwitch. A sua volta, il vSwitch instrada il traffico verso la rete fisica tramite le schede di rete reali del server (**pNIC**). Il vSwitch gestisce in software logiche complesse come tagging VLAN, traffic shaping e raggruppamento delle interfacce fisiche in modo del tutto trasparente per le macchine virtuali ospiti.

4.3.4 Live Migration

La funzionalità più dirompente della virtualizzazione è la **Live Migration** (es. VMware vMotion), che permette di spostare una VM accesa da un host fisico a un altro senza interruzione di servizio. Si sviluppa rigorosamente in tre fasi sequenziali:

1. **Pre-copy (Dirty tracking):** Trasferimento massivo iniziale della memoria mentre la VM gira sul vecchio host. Il sistema traccia attivamente tramite una bitmap

(*dirty page tracking*) quali pagine di memoria vengono modificate durante la lenta operazione di copia in rete. Questo ciclo viene ripetuto per trasferire i delta successivi.

2. **Stop and copy:** Quando le dirty pages rimaste sono pochissime, la VM viene "congelata" (pausa impercettibile di pochi ms). Viene copiato lo stato residuo dei registri CPU, i buffer di I/O, e la RAM residua. L'esecuzione viene trasferita all'hypervisor di destinazione.
3. **Gestione della Rete (Gratuitous ARP):** Poiché il MAC address della VM si è spostato fisicamente su un'altra porta dello switch core del datacenter, il vSwitch di destinazione lancia un **Gratuitous ARP** broadcast. Questo aggiorna istantaneamente le tabelle CAM di tutti gli switch fisici, garantendo che i pacchetti in transito non vengano persi.

4.4 Containerizzazione

A differenza della virtualizzazione, i **Container** non emulano un intero hardware o un intero kernel, ma **condividono il kernel Linux dell'host**. Questo riduce a zero l'overhead, ma cambia radicalmente le modalità di sicurezza e funzionamento in rete.

4.4.1 Meccanismi di Isolamento e Vulnerabilità

L'isolamento, vitale per impedire che i container collassino tra loro, è delegato al kernel del sistema operativo tramite due primitive:

- **Namespaces:** Isolano la visibilità logica. Un namespace fa credere al container di essere l'unico sistema, dotandolo di un PID tree autonomo e mount point esclusivi.
- **Cgroups (Control Groups):** Limitano l'utilizzo hardware (impongono tetti massimi di CPU o ferrei *OOM killer* per la RAM).

Condividere il kernel comporta però che una falla in esso colpisca tutti simultaneamente. Il corso cita l'anomalo caso della vulnerabilità **CVE-2026-31431 ("Copy Fail")** nel modulo `authencsn`: abusando del *page cache* condiviso tra container, un banale script Python permette non solo la scalata di privilegi, ma un catastrofico *container escape*, mettendo a repentaglio interi nodi di orchestrazione.

4.4.2 Dettagli Architettureali Docker

- **Overlay File System:** I container usano file system a strati (es. *overlay fs*). L'immagine del container è immutabile (sola lettura). Avviandolo, si crea un leggero layer *Read-Write* effimero in cima. Modificare un file base scatena una lenta *Copy-on-Write* verso il layer superiore.
- **Multi-stage Build:** Nello sviluppo software moderno, si usa un container gigantesco per compilare il codice sorgente. Poi, un secondo stage copia solo il binario finale compilato dentro un'immagine vuota (es. Alpine Linux da 5MB), riducendo drasticamente impronta e superficie d'attacco.

- **Driver Hardware distribuiti via Container:** Perfino i driver complessi si stanno "containerizzando". NVIDIA, per facilitare i deployment AI sui nodi nudi, distribuisce l'NVIDIA Container Toolkit, incapsulando driver GPU e librerie CUDA in un container isolato anziché installarli sull'host.

4.4.3 Network: VM vs Container

Nelle macchine virtuali, l'hypervisor crea una **vNIC** isolata (una vera scheda finta visibile alla VM) e la connette al vSwitch. Nei container, questa astrazione manca. I container istanziati (se non usano bridge complessi come in K8s) **contendono direttamente l'accesso alla pNIC** (scheda fisica dell'host) tramite routing interno. Proprio per questo, i container necessitano del **Port Mapping**: la porta interna del demone (es. la 80 di NGINX) deve essere mappata e traslata tramite NAT su un'altra porta esposta del server host.

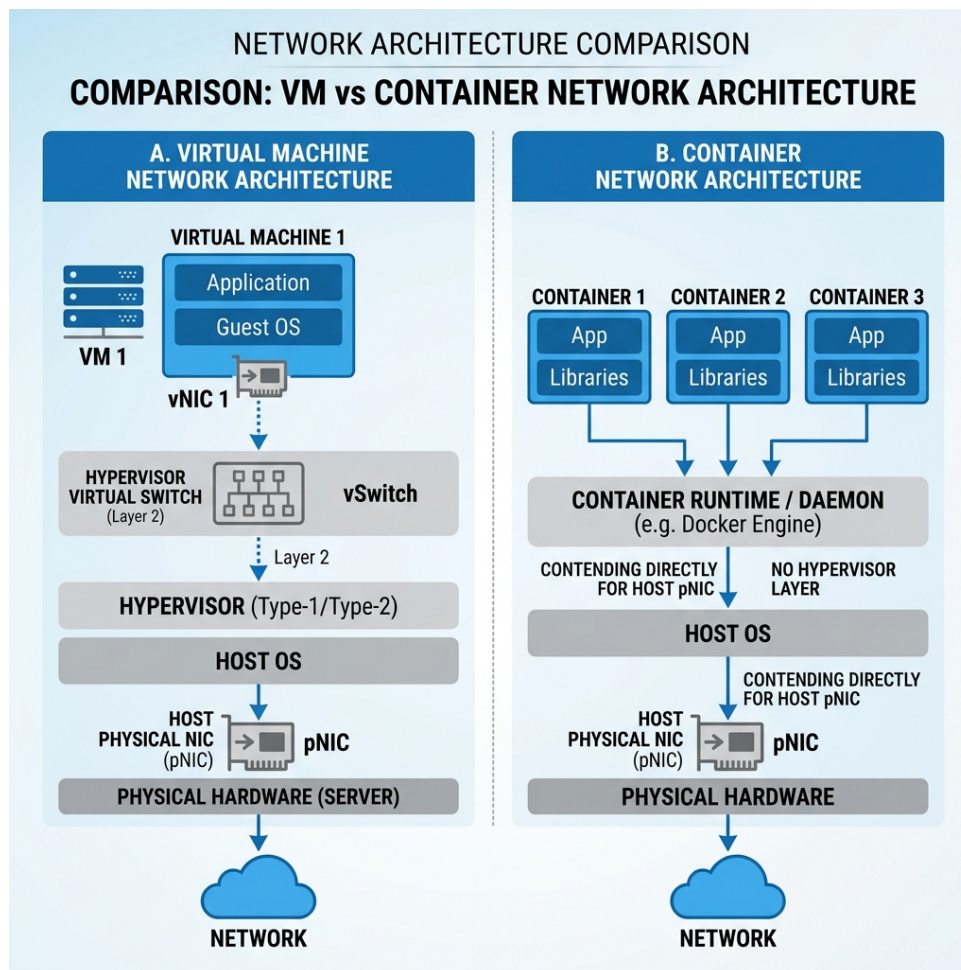


Figura 4.5: Architettura di rete: astrazione vNIC (VM) vs contesa e NAT (Container).

L'integrazione di Container e Virtualizzazione è però lo standard: il cloud esegue spesso cluster Kubernetes composti da *Worker node* che sono essi stessi macchine virtuali, combinando così la leggerezza e facilità di deployment del container con il robusto isolamento hardware (evitando container escape) della VM.

Capitolo 5

Cloud Computing, Orchestrazione e Sicurezza

5.1 Il Modello Cloud Computing

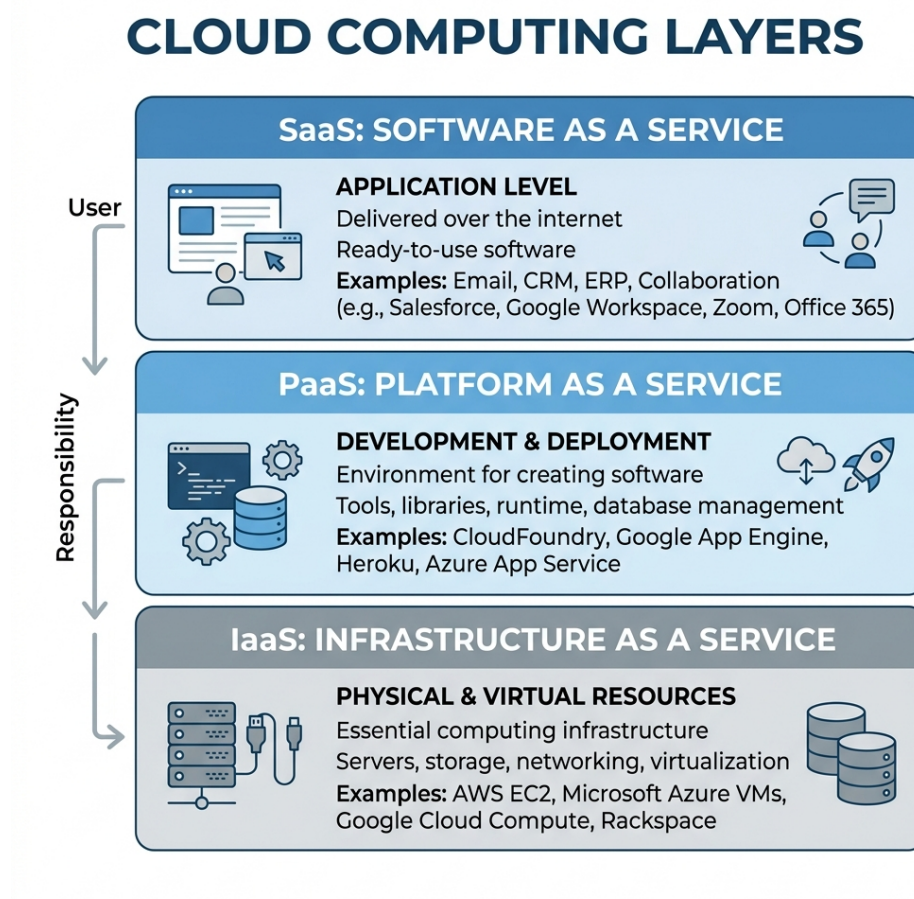


Figura 5.1: Rappresentazione astratta dei layer dell'architettura Cloud Computing.

Il cloud computing rappresenta non solo un cambio tecnologico, ma un profondo mutamento nel modello di business IT: si passa dall'acquisto di hardware (CapEx) al pagamento per consumo di un servizio (OpEx).

Le Sfide della Pubblica Amministrazione

Costruire un cloud per un'azienda nativa digitale o per la Pubblica Amministrazione (PA) sono due sfide opposte. Una startup opera in ambiente **Greenfield** (si parte da zero con standard moderni). La PA opera tipicamente in **Brownfield**: deve migrare nel cloud applicativi e mainframe vecchi di decenni senza interrompere servizi critici ai cittadini. Inoltre subentrano ostacoli strutturali:

- **Governance:** Modello centralizzato (più sicuro ed economico su scala nazionale) vs Federato (politicamente preferito dagli enti locali che non vogliono cedere il controllo sui propri dati).
- **Chargeback:** Come ripartire esattamente i costi (fatturazione interna) tra dipartimenti/ministeri diversi che condividono lo stesso hardware senza generare contenziosi.
- **Procurement:** L'acquisto di hardware fisico nei cloud on-premise della PA richiede estenuanti gare d'appalto pubbliche (con ritardi medi di 1-2 anni), rendendo il server acquistato già tecnologicamente obsoleto nel momento in cui viene fisicamente consegnato nel datacenter.

5.1.1 Definizione e Caratteristiche (NIST)

Secondo il NIST, il cloud è caratterizzato da:

- **On-demand self-service:** Provvigionamento autonomo da parte del cliente senza intervento umano del provider.
- **Broad network access:** Accessibilità ubiqua via rete standard.
- **Resource pooling:** Risorse condivise in modalità multi-tenant.
- **Rapid elasticity:** Capacità di scalare rapidamente verso l'alto e verso il basso (le risorse appaiono infinite al consumatore).
- **Measured service:** Modello pay-as-you-go e misurazione continua delle risorse.

5.1.2 Modelli di Servizio

- **IaaS (Infrastructure as a Service):** Il provider fornisce CPU, rete e storage "grezzi" o virtualizzati. Il consumatore gestisce dal Sistema Operativo in su.
- **PaaS (Platform as a Service):** Il provider gestisce anche l'OS e l'ambiente di runtime. Il consumatore carica solo la sua applicazione (es. database managed, web app).
- **SaaS (Software as a Service):** Tutto è gestito dal provider (es. Gmail, Microsoft 365). L'utente usa solo il software finale.

5.2 Architettura di Riferimento (Reference Model)

Il Reference Model si articola in livelli (layer):

1. **Physical Layer:** Hardware fisico (Server, Switch, Storage array). A questo livello operano gli **Element Manager**, ovvero interfacce di gestione proprietarie fornite dal produttore (es. la GUI di un singolo switch o di un array EMC) che vedono *solo* il proprio dispositivo fisico.
2. **Virtual Layer:** Macchine virtuali, vSwitch, LUN virtuali. Astrae l'hardware fisico fornendo pool di risorse flessibili.
3. **Control Layer:** Si occupa di gestire e raggruppare globalmente le risorse virtuali. Qui opera lo **Unified Manager** (es. VMware vCenter o Microsoft SCVMM), un super-gestore che aggrega migliaia di host ignorando le differenze di brand hardware e permette all'amministratore di applicare *policy* di allocazione (riserve assolute vs relative) e di sovra-allocazione (Overbooking).
4. **Orchestration Layer:** Automatizza flussi di lavoro complessi tramite script e chiamate asincrone ad API. Mentre il control layer esegue un singolo compito primario ("accendi VM"), l'orchestratore coordina una *pipeline* intera multi-step ("crea VM, configura vSwitch, alloca IP, installa OS, installa Database").
5. **Service Layer:** Fornisce il *Cloud Portal* e il *Service Catalog* in modalità self-service, da cui gli utenti finali (tenant) ordinano le risorse con un click, senza dover aprire ticket all'IT.

5.2.1 Cloud Service Lifecycle

Dietro le quinte, l'erogazione di un servizio cloud passa attraverso quattro macro-fasi obbligatorie:

- **Phase 1: Service Planning.** L'architetto IT definisce i confini del servizio, calcola le risorse richieste e, soprattutto, definisce la **Billing Policy** (la metrica esatta di fatturazione, es. per GB allocato, per CPU-hour o per Giga di traffico in uscita).
- **Phase 2: Service Creation.** L'orchestratore riceve l'ordine dal portale e lo converte in dozzine di chiamate API concrete all'infrastruttura per costruire il servizio, allocando IP, dischi e CPU.
- **Phase 3: Service Operation.** Il servizio entra in produzione e passa in mano al tenant. Viene costantemente monitorato tramite sonde per verificare il rispetto degli SLA (Quality of Service) e per quantificare le risorse consumate a fini di addebito (Metering).
- **Phase 4: Service Termination.** Alla scadenza del contratto o disdetta, il sistema deve automaticamente disallocare tutte le risorse, cancellare i dati in modo irreversibile (Secure Wipe) per proteggere la privacy del vecchio tenant, e rimettere indirizzi IP e storage nel pool disponibile.

SLA (Service Level Agreement) e Clausole Legali

È il contratto vincolante tra cloud provider e consumatore che stabilisce i livelli di servizio garantiti (uptime, performance, security). Include metriche di misurazione e sanzioni (*Service Credits*) in caso di violazione. Storicamente, gli SLA nascondono clausole complesse a favore del provider:

- **Downtime vs User-minutes:** Il calcolo del disservizio non è assoluto. Un guasto del 10% dell'infrastruttura potrebbe non violare l'SLA se calcolato sul totale degli "user-minutes" mensili previsti.
- **Esclusioni:** Le manutenzioni programmate (*Scheduled Downtime*) o i disservizi causati da un uso improprio del cliente esulano dalle sanzioni.
- **Lock-in:** Difficoltà tecnico-legali nel migrare i propri dati in uscita senza incorrere in enormi costi di trasferimento (*Egress*) o dipendenza da formati proprietari.

5.3 Business Continuity e Data Protection

Il cloud assume che i guasti hardware siano fisiologici. L'obiettivo è minimizzare l'impatto sul servizio (High Availability). I parametri principali sono:

- **RTO (Recovery Time Objective):** Tempo necessario per ripristinare il servizio in caso di disastro.
- **RPO (Recovery Point Objective):** Quanto indietro nel tempo si perde in dati in caso di ripristino. Determina la frequenza dei backup.

5.3.1 Tecniche di Resilienza e Limiti Geografici

Per evitare i **SPOF (Single Point of Failure)**, si usano tecniche di ridondanza su più livelli:

- **Failover di Servizio vs Replicazione VM:** Il *Service Failover* tradizionale (es. Windows Server Failover Cluster) si basa su un nodo secondario passivo; se il primario muore, il secondario monta lo storage condiviso e fa un lento reboot a freddo dell'applicativo (con RTO di minuti). La *Replica VM* o la **Continuous Data Protection (CDP)** operano invece a un livello inferiore. Un CDP inietta un driver (*Write Splitter*) nell'hypervisor che biforca ogni scrittura I/O: una copia va al disco locale, l'altra via rete asincrona al sito di Disaster Recovery. Utilizza un *Journal Volume* per mantenere una cronologia granulare al secondo, permettendo di "riavvolgere" nel tempo l'intera VM a 5 secondi prima di un eventuale attacco ransomware.
- **Active/Active vs Active/Passive:** L'approccio Active/Passive usa un nodo secondario in standby. L'approccio Active/Active distribuisce il carico simultaneamente su più datacenter geografici, ma incontra un grave limite fisico dettato dalla velocità della luce nella fibra ottica.

- **Replica Sincrona vs Asincrona:** Affinché due datacenter operino in vero Active/Active (senza corrompere i database condivisi), la replica dello storage deve essere **Sincrona** (la scrittura viene confermata al client solo dopo che entrambi i siti geografici l'hanno salvata). Per far ciò senza paralizzare le applicazioni, il tempo di latenza (*Round-Trip Time*) tra i datacenter non deve superare i **10-15 millisecondi**, il che obbliga i siti a non distare più di ~100-150 km l'uno dall'altro. Per distanze superiori (es. tra continenti), si è obbligati a usare la replica **Asincrona** (dove c'è una finestra di vulnerabilità, $RPO > 0$), poiché l'attesa del segnale luminoso renderebbe il sistema inutilizzabile.
- **Availability Zones:** Divisione in zone distinte (spesso in datacenter diversi entro la soglia dei 15ms) isolate da guasti comuni (alimentazione, allagamenti) ma connesse in fibra ottica dedicata (Dark fiber) per apparire come un'unica grande infrastruttura logica.

5.3.2 Application Resiliency: Progettare per il Fallimento

Le moderne architetture cloud si arrendono all'evidenza statistica che l'hardware fallirà inevitabilmente. La resilienza viene quindi astratta e spostata totalmente a livello software (**Design for Failure**):

- **Statelessness e Persistent State:** I container e i web server non devono *mai* salvare dati locali. Lo stato persistente viene scaricato costantemente su database esterni o Object Storage (es. S3), permettendo a Kubernetes di "uccidere" e ricreare un nodo applicativo infetto o guasto in 1 secondo, senza perdite e senza causare interruzioni agli utenti.
- **Graceful Degradation:** Se il sofisticato servizio di raccomandazione AI di Netflix fallisce nel cloud, l'interfaccia utente non deve generare una pagina di errore 500. Deve degradare l'esperienza in modo "elegante", servendo automaticamente una banale lista statica generica precompilata di film.
- **Retry Logic e Circuit Breaker:** Le chiamate API asincrone tra microservizi cloud falliranno per timeout di rete o latenza. Si implementano pattern software per ritentare (*Exponential Backoff*) o "aprire il circuito" (*Circuit Breaker*), disabilitando del tutto la chiamata problematica per evitare di bombardare e definitivamente abbattere un microservizio già congestionato.
- **Event-Driven Architecture:** Disaccoppiamento tramite code asincrone (es. Kafka o RabbitMQ). Se un database è temporaneamente sovraccarico per un picco di traffico imprevedibile, le scritture in arrivo si accumulano pazientemente in coda invece di fallire, venendo smaltite non appena il database torna a respirare.

5.4 Sicurezza (Security)

La sicurezza nel cloud si basa sul concetto di **Trust** tra cliente e fornitore, costruito tramite visibilità e controllo. L'obiettivo è garantire i pilastri **CIA** (Confidentiality, Integrity, Availability).

5.4.1 Minacce Classiche e Attacchi

Storicamente, i datacenter dovevano difendersi da attacchi mirati al perimetro (Firewall). Le minacce classiche includono:

- **Spoofing:** Falsificazione dell'identità (es. IP spoofing) per bypassare le ACL.
- **Man-in-the-Middle (MitM) e Sniffing:** Intercettazione non autorizzata del traffico in transito.
- **Denial of Service (DDoS):** Saturazione delle risorse (computazionali o di banda) per abbattere la disponibilità del servizio.
- **Phishing / Social Engineering:** Compromissione del fattore umano per rubare credenziali valide.

5.4.2 Gestione delle Identità e Policy (AAA e RBAC)

Oltre alla difesa perimetrale, il fondamento vitale della sicurezza cloud in ambienti multi-tenant è la corretta gestione degli accessi e dei privilegi:

- **Modello AAA:** Rappresenta il framework di sicurezza fondamentale.
 - *Authentication (Chi sei?):* Verifica rigorosa dell'identità (es. tramite password, certificati o Multi-Factor Authentication).
 - *Authorization (Cosa puoi fare?):* Valutazione e concessione dei permessi esatti dell'utente sulle singole risorse del cloud.
 - *Accounting (Cosa hai fatto?):* Tracciamento e logging immutabile di ogni azione (Audit Trail) per scopi di analisi forense post-incidente e per la fatturazione.
- **RBAC (Role-Based Access Control):** Nel cloud i permessi non si assegnano quasi mai al singolo utente direttamente (sarebbe ingestibile). Vengono creati dei **Ruoli** logici (es. "Storage Admin", "Auditor"). L'utente assume il ruolo o viene associato ad esso. Questo sistema evita la proliferazione incontrollata di privilegi e garantisce il rispetto del principio della sicurezza del *Least Privilege* (fornire sempre e solo il minimo privilegio necessario per compiere una mansione).

5.4.3 Approccio Defense-in-Depth e Zero Trust

Poiché gli attacchi classici hanno dimostrato che non esiste un solo perimetro sicuro e che le minacce provengono anche dall'interno (*Insider Threat*), il modello classico (basato su un'unica DMZ esterna) è oggi sostituito dalla **Zero Trust Architecture (ZTA)**. Il principio è: "Non fidarsi mai, verificare sempre", basandosi su autenticazione forte e *Micro-segmentazione* dinamica prima di accedere a ogni singolo workload. La base dell'architettura di sicurezza è la **TCB (Trusted Computing Base)**, l'insieme dei componenti critici (es. hypervisor, firmware BMC), che se compromessi fanno cadere inevitabilmente l'intera catena di fiducia.

5.4.4 Gestione dei Rischi (GRC)

Si racchiude nel framework **GRC** (Governance, Risk management, Compliance):

- Nessun sistema è inviolabile al 100%. Il Risk Assessment valuta la probabilità di incidente contro il costo della contromisura.
- La Compliance assicura il rispetto di normative come GDPR, NIS2, ecc.

Capitolo 6

Operations e Considerazioni Finali

6.1 Service Operations Management

Questa fase chiude il ciclo di vita del datacenter affrontando la parte procedurale e operativa: tutto ciò che avviene *dopo* che l'infrastruttura è in piedi. Mantenere funzionante un cloud significa garantire il rispetto degli **SLA (Service Level Agreement)**, identificare guasti, pianificare le risorse e tracciare ogni modifica. Le attività principali si raggruppano in:

- **Monitoring:** Osservare continuamente lo stato dell'infrastruttura e identificare anomalie.
- **Configuration e Change Management:** Tracciare l'inventario e governare ogni modifica al sistema.
- **Capacity Management:** Capire quante risorse serviranno nei prossimi mesi per approvvigionarle prima che si esauriscano.
- **Incident e Problem Management:** Rispondere a eventi non pianificati e prevenire la loro ricorrenza.

6.1.1 Monitoring e Predictive Failure

In un'infrastruttura moderna esistono milioni di contatori (rete, hypervisor, BMC). Poiché ogni sistema "pulito" genera migliaia di warning nei log, il vero lavoro del monitoring è separare i problemi reali dal rumore di fondo usando i dati storici. Si distinguono:

- **Configuration monitoring:** Rilevare modifiche non autorizzate o violazioni di policy.
- **Availability monitoring:** Verificare la raggiungibilità dei servizi e lo stato dei singoli componenti hardware (es. dischi).
- **Security monitoring:** Fondamentale nei cloud pubblici, monitora accessi logici e fisici (es. sensori biometrici alle porte).

Una funzionalità avanzata è la **Predictive Failure Analysis**: i BMC integrano modelli statistici e, analizzando i pattern di un disco, emettono un avviso *prima* che si rompa. Questo permette di rimuoverlo dal pool, sostituirlo e reinserirlo senza alcun impatto sull'utente.

Il clock della ridondanza e gli effetti a cascata

Quando un componente guasta, il "clock comincia a ticchettare": la ridondanza è ridotta e la sostituzione deve essere rapida. Un fallimento può inoltre generare un **effetto a cascata**: il carico spostato su altri nodi sani li sovraccarica, facendoli crollare a loro volta.

6.1.2 Capacity Monitoring e Management

Il Capacity Management ragiona come una compagnia assicurativa: si fanno stime statistiche e si scommette che non tutti useranno le risorse al massimo contemporaneamente. Questo permette la pratica dell'**Overbooking** (o *over-commitment* di CPU e rete). Alcune tecniche operative includono:

- **Dynamic scheduling**: Bilanciamento automatico del carico tra le vCPU dei server fisici.
- **Tiering dello storage**: Spostare i dischi delle VM spente su storage lento ed economico. Alla riaccensione, la VM viene avviata e lo storage subisce una live migration in background verso un tier veloce in modo totalmente trasparente per l'utente.

6.2 Gestione dei Cambiamenti e dei Guasti

6.2.1 Change Management

È il processo volto a registrare ogni singola modifica alla configurazione di un sistema nel **CMDb (Configuration Management Database)**. Quando qualcosa si rompe, la prima domanda del team operations è sempre "chi ha toccato qualcosa?". Qualsiasi modifica strutturale (es. aggiornamento DB) richiede un processo rigido: creazione di uno snapshot (garantendo la reversibilità in caso di rollback), aggiornamento, verifica e, solo se ha successo, eliminazione dello snapshot per non gravare sulle performance di I/O future a causa della catena dei file differenziali.

6.2.2 Incident vs. Problem Management

La distinzione ITIL tra incidente e problema è vitale per le operations:

Incidente vs Problema

Un **Incidente** è un evento non pianificato che degrada un servizio (approccio *reattivo*: si risolve il sintomo nel minor tempo possibile). Un **Problema** è la causa radice di uno o più incidenti (approccio *proattivo*: si elimina la causa per evitare che si ripeta).

Esempio pratico: un disco si rompe e un pool di storage si esaurisce (incidente). Cambiare il disco risolve l'incidente. Il *problem management* indaga e scopre che il pool era già troppo saturo, quindi la soluzione a lungo termine è espandere la capacità generale del cluster per prevenire guasti futuri.

6.3 Considerazioni Finali sul Datacenter

L'evoluzione dell'IT ha seguito un percorso che parte dalle basi fisiche (energia, dissipazione) fino all'astrazione totale (cloud). Alcuni concetti chiave riassuntivi:

- **Il principio del bilanciamento:** Un sistema efficiente non è quello con il miglior componente assoluto, ma quello bilanciato. Una CPU iperveloce con uno storage a dischi meccanici darà un sistema lento. Il design del datacenter è per sua natura un esercizio per evitare e gestire i colli di bottiglia.
- **Workload divergence:** Mentre un tempo l'hardware era "general-purpose", oggi i carichi divergono. L'Intelligenza Artificiale richiede enormi quantità di memoria, GPU densamente impaccettate e latenze di rete ultra-basse, portando a concentrazioni di potenza termica (fino a gigawatt in singoli edifici) e soluzioni estreme di direct-to-chip liquid cooling impensabili un decennio fa.
- **Architettare per il carico:** Di fronte a enormi flussi di dati (es. gli esperimenti del CERN che producono 10 TB/s ma ne scartano il 90% dopo una prima elaborazione), l'ingegnere deve dimensionare frontend e backend in modo asimmetrico, scegliendo l'infrastruttura (SAN vs Object/NAS) in base ai reali pattern di accesso, latenza accettabile e parallelismo richiesto.